AGENCY PLATFORM

Project Handbook

The way we build it, what you need to know and prepare before you start, the plan for the first slice, and a review of every concept that slice uses.

HOW TO USE THIS HANDBOOK

- **Parts A–C** (about one hour): read once now. They cover the client, the process, your machine, the decisions still open and the questions for the client.
- **Part D** is the checklist for slice 1, the walking skeleton, split into pull requests.
- **Part E** is reference. Read a concept card just before the task that introduces it. Every card answers the same questions: what it is, why we need it here, how it looks in this project, what to watch out for and where to read more. Cards marked Deep dive explain in more depth.

Coloured boxes: Recommendation a proposal that needs your decision (and an ADR when marked) · Ask the client · Gotcha a trap people often fall into · Check how to verify a step.

Version 1 · 11 October 2026 · prepared with Claude Code for the developer of the Agency Platform
Sources: `CLAUDE.md`, `START-HERE.md`, `docs/analysis-and-design.md`, `docs/development-strategy.md`, ADR 0001–0006, and a check of the development machine on 11 October 2026.
Code in this handbook is a *sketch* that shows the shape. The real code arrives in the pull requests, with tests.

CONTENTS

Contents

Part A · The project 3

A1. The client and the problem 3

A2. Business vocabulary 4

A3. Your two roles: delivering for a client, and learning 5

A4. Decisions already made 6

Part B · The process 8

B1. The roadmap: seven phases, each with a gate 8

B2. How one vertical slice is built 8

B3. Your daily loop 9

B4. Definition of done 10

B5. Writing an ADR 10

B6. Keeping documentation alive 10

Part C · What you should know and prepare 12

C1. Your machine (checked 11 October 2026) 12

C2. Things found in the repository today 12

C3. Decisions to make during slice 1 13

C4. Questions for the client 15

C5. Costs and accounts to expect 16

Part D · Slice 1: the walking skeleton 17

D1. Goal and finish line 17

D2. Repository layout after slice 1 18

D3. The tasks, one pull request each 18

D4. Task by task 20

Part E · Concept reviews 28

E1. Architecture and ways of working 29

E2. C# and .NET 34

E3. Data: EF Core and PostgreSQL 40

E4. Security 43

E5. Front end 46

E6. Containers, Kubernetes and delivery 49

E7. Git and pull requests 53

Appendix · Reference 54

F1. Acronyms 54

F2. Official documentation 54

F3. What comes next (slice 2 preview) 55

PART A

The project

Who the client is, what is wrong with what they have, what we are building instead, and the business words you will hear every day.

A1. The client and the problem

The client is a **DMC** (Destination Management Company) based in Morocco, with branches in **Agadir** and **Marrakech**. A DMC runs the "ground" part of a trip. It buys hotel nights, transfers, excursions, guides and vehicles from local suppliers, packages them (for example as circuits), and sells them to foreign tour operators, travel agencies, online travel agencies (OTAs such as Expedia) and directly to travellers. It earns the **margin** between what it buys at (the *cost*) and what it sells at (the *sale price*).

Today the agency runs "**Agence 2024**", a Windows desktop application.

Old system	What it means for us
VB.NET WinForms, .NET Framework 4.8, ~140 forms	Business logic sits in click handlers and is copied between forms. Nothing can be reused.
SQL Server 2017: 127 tables, 780 stored procedures	The rules live in procedures. About 40 procedures that the code calls do not exist.
112 of 127 tables have no primary key, and there are no foreign keys	Duplicates and orphan rows are possible. This is why the new system starts empty (ADR 0005).
One database per season (`agence_sti_2026`)	Reports across years and contracts that cross November need workarounds.
Crystal Reports, 93 `.rpt` files	Replaced by a PDF library in the new system.

The five problems that drive the whole project

1. **Contracts are not a price source.** Several versions of the same contract circulate, so prices are typed and checked by hand. The client wants prices **calculated automatically**, and that only works if exactly one version is valid on any date.
2. **Invoicing is unsafe.** Issued invoices get overwritten, cancelled numbers get reused, and two operators can receive the same number because the code reads the counter and then increments it in two separate steps.
3. **Money is stored as `float`**, which causes rounding errors on totals. There is no real VAT. Closed months can be reopened by anyone.
4. **No itinerary.** A booking file is a list of sales lines, so vouchers, transfers and lists are retyped by hand.
5. **Cash is unsafe.** Some procedures corrupt data (an `UPDATE` without a `WHERE` recalculates every cash voucher). There is no bank reconciliation and no closing. Passwords sit in plain text in `app.config`.

Each problem becomes a rule in `CLAUDE.md`. Point 1 becomes "one active contract version per date". Point 2 becomes "invoice numbers change only inside a transaction with a row lock". Point 3 becomes "money is `decimal` / `numeric(18,2)`". Knowing the reason behind each rule lets you defend it when a shortcut looks tempting.

What we build instead

We build **three front ends on one backend**. All of them get prices, availability and bookings from the same rules and the same database.

Front end	Who	Main features
Back office	Agency staff	Catalogue, supplier and sales contracts (with AI intake), bookings and itineraries, billing, treasury, reports, users
Partner portal + Partner API	Tour operators, agencies, OTAs	Search, net prices, book, cancel, vouchers, invoices. API with webhooks. Booking emails read by AI into drafts
Public site	Travellers	Catalogue, availability, indicative prices, quote requests, AI guest assistant (web and WhatsApp)

Managers also get an **MCP server**, the "manager assistant". It lets an AI client answer questions such as "what did Agadir sell last month?" using read-only tools.

A2. Business vocabulary

You cannot model a domain you cannot talk about. These words appear in the client's mouth, in the old database (often in French) and in our code (in English).

Term (French in old system)	Meaning
Contract — buy (*achat*) / sell (*vente*)	A **supplier contract** sets what the agency pays a hotel or other supplier. A **sales contract** sets what a client (tour operator, market) pays the agency. In the new system both are versioned, with one active version per date.
Season (*saison*)	A named set of date ranges with the same price level, for example "High season: 20 Dec–5 Jan, 1 Apr–30 Apr". The old system allowed at most 5–6. The new one allows any number.
Rate (*tarif*)	A price for a room type + board + occupancy in a season, per night or per person.
Room type, board	Room type examples: Double, Single, Suite. Board is the meal plan: RO (room only), BB (bed & breakfast), HB (half board), FB (full board), AI (all inclusive).
Pax, occupancy	"Pax" means persons. Occupancy is how many adults and children (with ages) share a room.
Supplement / reduction	Extra or discount rows: single supplement, 3rd adult reduction, child reduction by age range, gala dinner.
Promotion	Early booking (book before a date, get X%), "stay N pay M", long stay, honeymoon. Has stacking ("combinable") rules.
Allotment (*allottement*)	Rooms a hotel reserves for the agency per date. **Fixed** rooms are paid even if unsold. **Elastic** rooms are available on request. **Release days**: unsold rooms go back to the hotel X days before arrival.
Stop sale	The hotel says "no more sales for these dates", even inside allotment. The old system had no such concept.
Net rate, markup, commission	A net rate is the price a partner pays, before it adds its own margin. A markup is the agency's margin added on top of cost. A commission is a % the agency gives away.
File (*dossier*)	The centre of operations: one trip for one client, with its bookings, itinerary, documents and invoices.

Booking, service line	A booking is a client's reservation. A service line is one bookable service (hotel stay, transfer, excursion) with sale price and cost.
Circuit, itinerary	A circuit is a pre-configured multi-day tour (template). When it is booked, the template becomes the file's own day-by-day itinerary with real dates.
Voucher	A document sent to a supplier: "please provide these services to these guests, we pay". The supplier invoices against it.
Transfer	A transport leg: airport → hotel, hotel → hotel, hotel → airport. Has a pick-up time, vehicle type and driver or transport supplier.
Rooming list	A list per hotel and arrival date of who sleeps in which room.
Tour operator (TO), OTA	Foreign companies that sell the agency's services to travellers. OTA = online travel agency (Expedia, Booking).
Proforma, invoice, credit note (*avoir*)	A proforma is a quote-like document. An invoice, once issued, is immutable. A credit note is the only way to correct an issued invoice.
Allocation (*lettrage*)	Matching a payment to the invoices it pays.
Aged balance (*balance âgée*)	What each client owes, grouped by age (0–30, 31–60, 60+ days).
Cash session, folio, débours	A cash session is one day of a cash register, from opening to counting to closing. A folio is the old name for a group of cash vouchers. Débours are expenses advanced, for example by a guide.
Rep, ticket book (*carnet*)	Representatives sell excursions in hotels from paper ticket books and hand the cash in to the cashier.
MAD	Moroccan dirham, the agency's base currency. Contracts can also be in EUR, USD or GBP.
Sage	The accounting software that receives an export from our system.

A3. Your two roles: delivering for a client, and learning

This project has two goals that sometimes pull against each other. The client pays for working software. You also want to come out of it as a much stronger engineer. The plan in `docs/development-strategy.md` builds learning into delivery ("learn just in time"). These habits keep the two goals aligned:

1. **Estimate with learning included.** The first slices will be slow because every concept is new. Later slices reuse the same path and go faster. Tell the client this up front, then show it with the bi-weekly demos.
2. **Demo every two weeks on the test environment**, using their real contracts and circuits as examples (not as data in the code). A demo catches wrong assumptions while they are cheap to fix. Forward Deployed Engineers live by this habit.
3. **Write decisions down (ADRs).** When the client asks "why did you do it this way?" six months from now, the ADR answers, and it protects you too.
4. **Keep the client's assets in the client's name.** The cloud subscription, domain names, AI provider account, WhatsApp business account and, ideally, the GitHub organisation should belong to the client, with access granted to you. Agree in writing who owns the code. (This is good practice, not legal advice. Check your contract.)
5. **Never put real data in code, tests or prompts.** Use invented hotels and guests. Guest data falls under Moroccan Law 09-08 (CNDP).
6. **Use Claude Code as a tutor, not a replacement.** Ask it to explain a concept with an example from this code before it writes the code. Write or review the domain test yourself. Read every diff and ask "why this

way?". Let it do the repetitive work (mappings, Helm boilerplate, form plumbing) and spend your own attention on pricing, versions, billing and security.

7. **Explain to learn.** After applying a concept, write 5–10 lines in `docs/learning/<concept>.md` in your own words, with a link to the code. Reread it a month later.

A4. Decisions already made

These decisions are recorded as ADRs in `docs/adr/`. Do not contradict them. To change one, write a new ADR that supersedes it.

ADR	Decision	Main reason	Rejected
0001	Modular monolith with Clean Architecture; each host is its own container	Small team, tightly connected rules; microservices add cost with no benefit	Microservices; layers without module borders
0002	PostgreSQL only (EF Core + Npgsql), pgvector, managed in production	No licence; `daterange` + exclusion constraints; `jsonb`; vectors in the same DB	SQL Server; separate vector DB
0003	React + Vite for back office and portal; Next.js for public site; generated API client	Data-heavy screens; static files on Kubernetes; server rendering for SEO	Blazor Server
0004	Managed Kubernetes in Europe near Morocco; managed PostgreSQL and Redis; Helm; GitHub Actions	No cluster/DB operations for a small team; backups and point-in-time recovery	Self-managed cluster
0005	Start with an empty system; no import from the old one	Old data has no keys, duplicate contract versions, float amounts	Importer from SQL Server
0006	AI drafts, humans approve; AI account cannot call approve endpoints	Contracts and money need a responsible person; AI can be wrong or manipulated	Auto-approval above a confidence score

The rules card (from CLAUDE.md)

ARCHITECTURE

- Layers: Domain → Application → Infrastructure → hosts. Dependencies point inward.
- Domain uses no framework (no EF Core, no ASP.NET). Application never references Infrastructure.
- Modules are folders in each layer. They talk through application interfaces or domain events only.
- Rules live in entities and value objects.
- One repository per aggregate. No generic repository, no MediatR or AutoMapper.
- Expected failures return a `Result`. Exceptions are for bugs and infrastructure.
- Architecture tests enforce all of this.

DATA

- Money: `decimal / numeric(18,2)`. Rates: `numeric(18,6)`. Never float.
- Stay dates: `DateOnly / date`. Moments: `DateTimeOffset / timestamptz` in UTC. Use `TimeProvider`, never `DateTime.Now`.
- Primary keys, foreign keys, and check, unique and exclusion constraints everywhere.
- snake_case names.
- Invoice numbers, allotment and payments change only in transactions with row locks, never from cache.
- Contracts, circuit templates and vouchers are versioned.

AI

- AI creates drafts only. A logged-in human approves, and the API enforces it.
- The guest assistant sees published, public data only.
- MCP tools are read-only. No free-form SQL.
- Every AI feature has an eval set in `tests/evals/`.

WORKING

- Thin vertical slices: domain test first.
- One change = one pull request.
- Run `dotnet test` and front-end tests before calling a task done.
- Ask before adding a dependency or project, or changing an ADR.
- No real data or secrets anywhere.

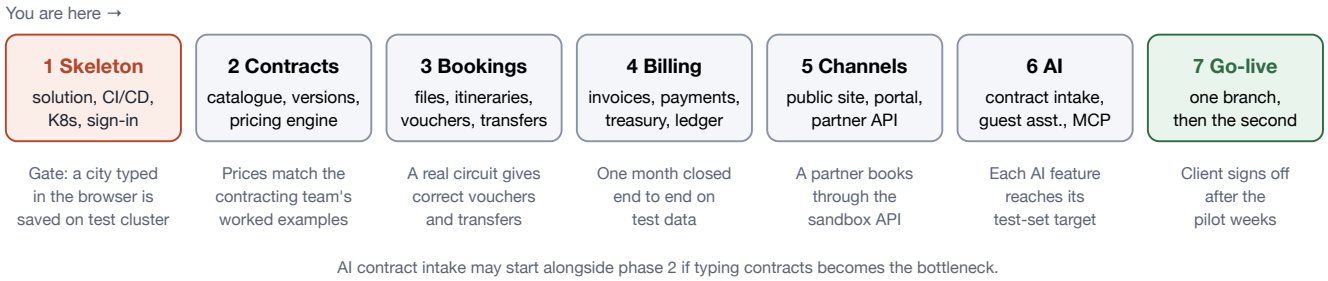
PART B

The process

How the work is ordered (phases and gates), how one slice is built, what a working day looks like, and when a task counts as done.

B1. The roadmap: seven phases, each with a gate

The phases follow the dependencies between modules. You cannot price without contracts, book without prices, or invoice without bookings. A phase ends only when its **gate** can be shown to the client on the test environment. "Code complete" does not count.



Inside phase 1 the first slice is the **walking skeleton**: "create a city and list cities". The business logic is trivial, but the slice crosses every layer and is deployed to Kubernetes. Infrastructure problems (sign-in, migrations, containers, networking) then show up in week one rather than at go-live. Part D plans this slice in detail. The second slice, the first real one, is a **hotel contract with one season and one rate, and the pricing engine returning a price for a stay**.

B2. How one vertical slice is built

A **vertical slice** is one feature built through every layer, from database to screen to deployment, before the next feature starts. The alternative, "horizontal" layering (all tables first, then all services, then all screens), delays the first working feature by months and hides integration problems until the end.

Inside each slice the order is fixed:

#	Step	What you produce	Why in this order
1	Domain test	A failing unit test that states the business rule ("a city name cannot be empty")	Forces you to decide the rule before writing code. Runs in milliseconds.
2	Domain code	Entity / value object that makes the test pass	The rule lives in one place, with no framework.
3	Application	Use case (command or query) + interfaces it needs, tested with fakes	Orchestration without a database.
4	Infrastructure	EF Core mapping, migration, repository; integration test against real PostgreSQL	Proves constraints and mapping.

5	API	Endpoints, permissions, OpenAPI, logs	Thin: it only translates HTTP to use cases.
6	React screen	Page using the generated API client	The user can see it.
7	Helm values	Config / secrets / routes the feature needs	It runs on the test cluster.

B3. Your daily loop

1. **Pick one small task** from the current slice, half a day to two days of work.
2. **Create a branch** from an up-to-date default branch.
3. **Explain first.** If the task introduces a new concept, read its card in Part E and ask Claude Code to explain it with this code.
4. **Write the domain test** (or review the one Claude proposes), then the code.
5. **Run all tests locally**, then commit with a message that says *why*.
6. **Push and open a pull request.** CI runs build, unit, integration and architecture tests.
7. **Review your own diff** against the definition of done (B4), then merge.
8. CD **deploys to the test environment** once it exists (end of slice 1). Check the feature there.

```
# one task, start to finish
git switch master && git pull
git switch -c feat/city-domain              # branch name: type/short-topic
# ... write test, then code ...
dotnet test backend/Agency.slnx               # all tests green
git add -p                                    # stage hunk by hunk: you read every change
git commit -m "Add City entity with name rules"
git push -u origin feat/city-domain
gh pr create --fill                           # opens the pull request on GitHub
gh pr checks --watch                          # wait for CI
gh pr merge --squash --delete-branch          # one clean commit on master
```

SIZE OF A PULL REQUEST

Aim for something you can review in 15 minutes: usually under ~400 changed lines, excluding generated files (migrations, API client). If a PR grows past that, split it. Example: "domain + tests" in one PR, "EF mapping + migration" in the next.

Working with Claude Code on a task

This loop comes from `START-HERE.md`:

1. "Explain <concept> with an example from this code" (before any code).
2. You write or review the domain test.
3. Claude implements. You read the diff and run the tests.
4. Ask "why this way?" about anything unfamiliar, and put the answer in a learning note.
5. Commit, push, open the PR, merge, and move to the next task.
6. If a decision was made, ask for an ADR in the same PR.

Conversations are not memory. The repository is. Anything that must survive a session goes into `CLAUDE.md`, an ADR, `docs/analysis-and-design.md` or a learning note.

B4. Definition of done

A task is done only when every box is ticked. Each item says how to check it.

- ☐ **Tests pass**: `dotnet test backend/Agency.slnx` and `pnpm -C frontend -r test` are green locally and in CI, including architecture tests.
- ☐ **No warnings**: the build treats warnings as errors (`TreatWarningsAsErrors` in `Directory.Build.props`), so a warning fails the build. Same for the TypeScript type check and lint.
- ☐ **OpenAPI updated**: new or changed endpoints appear in the OpenAPI document with request/response types and error codes, and the generated front-end client is regenerated.
- ☐ **Permissions checked**: every endpoint has an authorization requirement. An integration test shows that an anonymous call gets `401` and a user without the role gets `403`.
- ☐ **Meaningful logs**: important actions log one structured line with IDs (no personal data, no secrets).
- ☐ **Works on the test cluster**: after deployment, you have clicked through it there.
- ☐ **ADR if a decision was made**, and `analysis-and-design.md` updated if the design changed.

B5. Writing an ADR

An **Architecture Decision Record** is a short file that records one important decision: what was decided, why, and what was rejected. It is cheap to write and saves hours of "why is it like this?" later. Write one whenever you choose between real alternatives that would be costly to change: a library, a pattern, a hosting choice, a security approach. Do not write one for code style.

```
docs/adr/0007-guid-v7-primary-keys.md

# ADR 0007: GUID v7 primary keys

Status: Accepted · Date: 2026-10-20

## Decision
Entities use Guid v7 identifiers generated in the application (Guid.CreateVersion7()),
stored as PostgreSQL uuid.

## Why
The ID exists before saving (useful for events and links); IDs cannot be guessed or counted
by partners; v7 is time-ordered, so inserts stay index-friendly unlike random v4.

## Rejected alternatives
bigint identity columns (exposes counts, needs a DB round trip for the ID); Guid v4 (random index
order).
```

To change a decision, add a new ADR with `Status: Accepted · Supersedes 0007` and set the old one to `Superseded by 00xx`. Never edit history.

B6. Keeping documentation alive

- When the design changes, update `docs/analysis-and-design.md` **in the same PR** as the code.
- When a useful command appears (run tests, start services, deploy), add it to the "Commands" section of `CLAUDE.md`.

- When a client question is answered, tick it in the open-questions list and record the answer (with the date and who answered).
- This handbook is generated from `docs/handbook/handbook.html`. Update it at the end of each slice with the concepts that slice introduced.

PART C

What you should know and prepare

The state of your machine and repository today, what the repository docs leave out, the decisions still open for slice 1, and the questions to put to the client.

C1. Your machine (checked 11 October 2026)

Tool	Status	Notes / how to install
.NET SDK	✅ 10.0.302	.NET 10 is the current LTS (released November 2025, supported until November 2028). This is the "current .NET LTS" that `CLAUDE.md` asks for. Pin it with a `global.json` in task 0.
Docker	✅ 29.5.3	Docker Desktop must be *running* for `docker compose`, Testcontainers and kind.
Node.js	✅ v26.3.1	Node 26 is the "Current" release line and is scheduled to become LTS at the end of October 2026, so it is fine to keep. Pin the version in `frontend/package.json` (`engines`) and `.nvmrc` so CI uses the same one.
pnpm	❌ missing	`brew install pnpm` or `npm install -g pnpm`. Recent Node versions no longer bundle Corepack, so the old `corepack enable` route may not be available.
git, gh	✅ 2.54 / 2.96	Run `gh auth login` once, then `gh repo create` to create the private GitHub repository.
kubectl	✅ 1.34	OK.
kind	❌ missing	`brew install kind`. Needed from task 11 (local Kubernetes).
Helm	❌ missing	`brew install helm`. Needed from task 11.
k9s (optional)	—	`brew install k9s`. A terminal UI to look inside a cluster; very helpful while learning.
DB client	—	DBeaver or pgAdmin, to look at tables, constraints and data yourself.
Claude Code + VS Code	✅	Also install the C# Dev Kit extension in VS Code (or use Rider).

C2. Things found in the repository today

Finding	Why it matters / what to do
No Git remote yet; one commit on `master`	Create the private GitHub repository and push before task 2 (CI needs it).
Default branch is `master`, but the docs say "every merge to **main** deploys"	Choose one name before writing the CI workflow, because the workflow triggers on the branch name. Recommendation: rename to `main` (`git branch -m master main`, then set it as default on GitHub) so the docs stay true.

`._*` files next to every file	macOS writes these "AppleDouble" files when the project sits on a drive that is not formatted as APFS/HFS+ (your `/Volumes/saved`). Add `._*` and `.DS_Store` to `.gitignore` so they are never committed. Consider keeping the repository on the internal disk: Docker bind mounts and file watchers (Vite, `dotnet watch`) are faster and more reliable there.
`CLAUDE.md` says `dotnet test backend/Agency.sln`	.NET 10 creates solutions in the new XML format `Agency.slnx` by default. Update the command in `CLAUDE.md` when the solution is created (task 1).
"Commands" section of `CLAUDE.md` is a placeholder	Fill it in as tasks add commands (compose, migrations, client generation, kind).
Design doc mentions diagrams "in the shared doc"	Those diagrams are not in the repository. If you have the shared doc, export the diagrams to `docs/diagrams/` so the repo is self-contained.

GOTCHA · INGRESS NGINX IS RETIRED

The design and the concept tables say "Ingress controller". The most common one, the community **ingress-nginx** controller, was announced as retiring in November 2025, with maintenance ending in March 2026. The Kubernetes **Ingress API** itself still works but is frozen. Its successor is the **Gateway API**. Many tutorials still use ingress-nginx, so do not copy them blindly. You still need to understand Ingress (card E6), but choose a maintained controller: your cloud's managed option, Traefik, or a Gateway API implementation. Record the choice in an ADR at task 11. *Check the current status of the project before you choose.*

C3. Decisions to make during slice 1

The docs leave these open. Each one is a real choice. `CLAUDE.md` says to ask before adding a dependency, so each library below needs your "yes", and the marked ones need an ADR. My recommendation is given with a short reason so you can disagree on the merits.

#	Decision	Options	Recommendation and why	ADR	Task
1	Default branch name	`main` / keep `master`	**Rename to** `main`, to match the docs and modern defaults.	—	0
2	Build settings	`Directory.Build.props`, Central Package Management, `global.json`	**Yes to all.** Nullable on, warnings as errors, one place for package versions. This is what makes "no warnings" enforceable.	—	1
3	Test libraries	xUnit v3 · assertions: built-in, Shouldly, FluentAssertions	**xUnit v3 with plain** `Assert`, optionally Shouldly. FluentAssertions v8+ needs a paid licence for commercial work.	—	1
4	Architecture tests	NetArchTest.Rules, ArchUnitNET	**ArchUnitNET** if it is still actively maintained (check its last release). NetArchTest is simpler but saw little maintenance for years. Either satisfies `CLAUDE.md`.	—	1
5	Input validation	.NET 10 built-in minimal-API validation, FluentValidation	**Built-in** (`AddValidation()` + data annotations on request records). No dependency. Business rules stay in the domain anyway.	—	6
6	Primary keys	Guid v7 (app-generated), `bigint identity`	**Guid v7** (`Guid.CreateVersion7()`). The ID is known before saving, IDs cannot be counted by partners, and v7 IDs are time-ordered so indexes stay efficient.	✔	3

7	Case/accent-insensitive names	ICU non-deterministic collation, `citext`, lower() index	**ICU collation** `und-u-ks-level1`. "Fès", "FES" and "fes" are then equal for unique checks. `citext` ignores case only, not accents.	✔	5
8	snake_case mapping	`EFCore.NamingConventions` package, manual names	**The package** (one line: `UseSnakeCaseNamingConvention()`). Writing every name by hand is error-prone.	—	5
9	Identity provider (who signs users in)	Keycloak (self-hosted), Microsoft Entra ID, ASP.NET Core Identity + OpenIddict, Auth0	**Keycloak in Docker for slice 1.** It is free, standard OpenID Connect, and also handles partners (client credentials) later. If the client's staff already use Microsoft 365, Entra ID may be better for staff in production. That is a client question (C4). Note that ASP.NET Core Identity alone stores users but does not issue OIDC tokens.	✔	8
10	Where browser tokens live	SPA with PKCE (tokens in the browser), BFF (server keeps tokens, browser gets a cookie)	**SPA + PKCE for slice 1**, as the docs plan, with tokens kept in memory or session storage. Then review a BFF before go-live, because current OAuth guidance for browser apps prefers it.	✔	8–10
11	Same origin for UI and API	Separate hosts (needs CORS), one host with `/api` path routing	**One host, `/api` routed to the API**, by the Vite dev proxy locally and the Ingress/Gateway in the cluster. No CORS configuration, fewer surprises.	—	9, 11
12	OpenAPI UI	Scalar, Swagger UI	**Scalar** (or Swagger UI). .NET 10 generates the document itself (`Microsoft.AspNetCore.OpenApi`). Only the viewer is a package. Enable it in development and test only.	—	6
13	Front-end API client	openapi-typescript + openapi-fetch, orval, Kiota	**openapi-typescript + openapi-fetch**: generated types plus a tiny typed `fetch`. You write the TanStack Query hooks yourself, which is better for learning. orval generates the hooks too, if you want less code later.	✔	9
14	UI component library	MUI (+ MUI X DataGrid), Ant Design, shadcn/ui + TanStack Table	**MUI or Ant Design**: both have strong tables and forms for a data-heavy back office. shadcn/ui gives more design control but more assembly work. Pick one and stay with it.	✔	9
15	Logging	Built-in `ILogger` JSON console, Serilog	**Built-in** for now. Add OpenTelemetry in phase 3 as planned.	—	6
16	Local Kubernetes	kind, k3d	**kind**, as the docs say.	—	11
17	Ingress / Gateway	Cloud's managed option, Traefik, Gateway API implementation	**Decide at task 11**, given the ingress-nginx retirement (box above). Prefer what the chosen cloud supports natively.	✔	11
18	Container registry	GitHub Container Registry (GHCR), the cloud's registry (e.g. ACR)	**GHCR** to start: free with GitHub, and CI can push with the built-in token. Move to the cloud's registry if the client prefers.	—	12
19	Cloud and test cluster	Azure AKS (doc's first choice), OVHcloud, Scaleway	**Ask the client first** (who pays, data location). ADR 0004 already fixes "managed, Europe, near Morocco". This choice picks the provider.	✔	12

C4. Questions for the client

These come from the open-questions list in `analysis-and-design.md`, plus new ones raised by slice 1, grouped by **when the answer is needed**. Ask the first group now.

ASK THE CLIENT · NEEDED DURING SLICE 1

1. **Who owns and pays for the accounts?** Cloud subscription, domain name, GitHub organisation, container registry. The test environment has a monthly cost from day one.
2. **Where may data be hosted?** Is storing guest data in an EU region acceptable under Law 09-08 / CNDP, or must it stay in Morocco? (ADR 0004 asks for this confirmation before choosing the region.)
3. **How do staff sign in today?** Do they have Microsoft 365 or Google Workspace accounts? This decides the production identity provider.
4. **Which language must the back office use?** French? Arabic? Several? This affects screens from slice 1 onward (i18n is cheaper from the start).
5. **Domain names** for the test and production environments (for example `backoffice.test.<domain>`).
6. **Cities:** only Moroccan cities, or also cities abroad? Is a city name unique on its own, or per country? (This defines slice 1's unique rule.)
7. Can they share the "**factmarge**" API/web project referenced in the old solution?

ASK THE CLIENT · BEFORE PHASE 2 (CONTRACTS AND PRICING)

- Which VAT scheme applies (on margin or on full price), and must VAT and tourist tax appear on client invoices?
- More than 5 seasons, weekday prices, minimum stays?
- Should stop sales and release dates block bookings automatically?
- Sales prices: mostly a markup on supplier rates, or negotiated net rates per tour operator? Are supplier promotions passed on by default?
- Minimum margin: at what level is it a warning, and when does it block?
- Who may approve contract versions: one person per branch, or a central contracting team?
- Are Agadir and Marrakech still the only branches, with separate number series?
- Can the contracting team provide **worked pricing examples**? These are the gate for phase 2 and your most valuable tests.

ASK THE CLIENT · LATER PHASES

- **Phase 3:** own vehicles and drivers, transport suppliers, or both? How many circuit templates, and who can describe each day? Send vouchers automatically once confirmed, or after an operator check?
- **Phase 4:** which banks, and can they export statements and import transfer files? Which payment amounts need a manager's approval? Do reps still use paper ticket books?
- **Phase 5:** which partners would connect by API first, and in what format (XML, OpenTravel)? Credit limits and payment terms today?
- **Phase 7:** switch date, which branch goes first, and are running files finished in the old system?
- **All phases:** which reports do staff and managers need daily, weekly and monthly?

C5. Costs and accounts to expect

None of these is needed on day one, but the client should not be surprised by them. Get current prices from each provider, because they change.

Item	From	Note
Managed Kubernetes cluster (test)	Slice 1, task 12	Smallest node pool is enough for test. Production comes later.
Managed PostgreSQL (test, then production)	Slice 1, task 12	Production needs backups and point-in-time recovery (ADR 0004).
Domain + DNS, TLS certificates	Slice 1, task 12	Certificates are free with Let's Encrypt via cert-manager.
GitHub (private repo, Actions minutes, GHCR)	Now	Free tiers are usually enough at first.
Managed Redis	Phase 3	Cache and Data Protection keys.
Object storage	Phase 2–3	Contract files, generated PDFs.
LLM API	Phase 6 (or earlier for contract intake)	Use a contract that excludes training on the client's data.
WhatsApp Business Platform	Phase 5–6	Verified business, dedicated number, per-message pricing.
PDF library licence	Phase 3	For example QuestPDF: check which licence tier applies to the client's revenue.

PART D

Slice 1: the walking skeleton

"Create a city and list cities", built through every layer and deployed. The business rule is tiny on purpose, so all your attention goes to the technical path that every later feature will reuse.

D1. Goal and finish line

DONE WHEN

A signed-in admin types a city name in the browser, on the **test cluster**, and the city appears as a row in PostgreSQL. Every later feature reuses this exact path.

The rule for the city comes from `development-strategy.md`: **the name is not empty and is unique**. This handbook adds two details you should confirm: names are trimmed and at most 100 characters long, and uniqueness ignores case and accents ("Fès" = "fes").

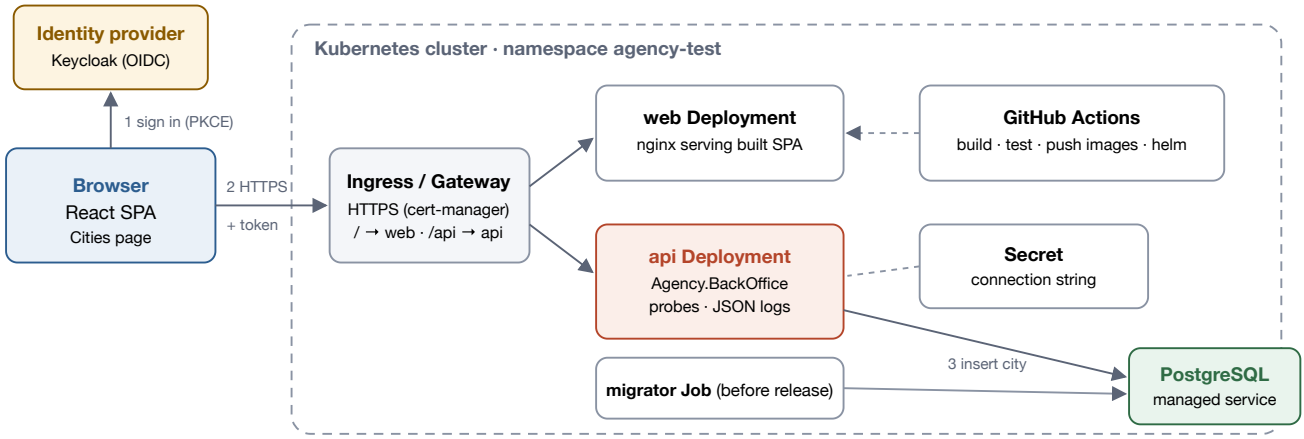


Figure D1 — The request path the walking skeleton proves. The API pod checks the token with the identity provider's public keys. PostgreSQL sits outside the cluster as a managed service.

D2. Repository layout after slice 1

```
agency/
  .github/workflows/ci.yml        build + test on every PR
  .github/workflows/cd.yml        images + deploy to test on merge
  backend/
    Agency.slnx
    Directory.Build.props         Nullable, TreatWarningsAsErrors, LangVersion
    Directory.Packages.props      every package version in one place
    global.json                   pins the .NET SDK
    src/
      Agency.Domain/Catalogue/City.cs  + Common/Result.cs, Error.cs
      Agency.Application/Catalogue/CreateCity.cs, ListCities.cs, ICityRepository.cs
      Agency.Infrastructure/Persistence/AgencyDbContext.cs, Configurations/, Migrations/
      Agency.BackOffice/Program.cs, Endpoints/CityEndpoints.cs
      Agency.Migrator/Program.cs
    tests/
      Agency.Domain.Tests/  Agency.Application.Tests/
      Agency.IntegrationTests/  Agency.ArchitectureTests/
  frontend/
    pnpm-workspace.yaml
    apps/backoffice/              Vite + React + TS
    packages/api-client/          generated from OpenAPI
  deploy/
    docker/                       Dockerfiles (or next to each project)
    helm/agency/                  Chart.yaml, templates/, values.yaml, values-test.yaml
    kind/                         local cluster config
  docker-compose.yml              PostgreSQL + Keycloak (+ later Redis, mail catcher)
  docs/ …
```

D3. The tasks, one pull request each

`development-strategy.md` lists 10 steps. Here they are split into pull requests and reordered slightly: CI comes right after the solution exists ("automate from day one"), and Kubernetes is split into local and cloud. Sizes are rough guesses for someone learning the tools, not commitments.

#	Pull request	Contents	New concepts (Part E)	Size
0	Repo housekeeping	GitHub remote, branch rename, `.gitignore`, folder layout, `.editorconfig`	Git & pull requests, ADR	½ day
1	Solution + architecture tests	Projects, references, build props, CPM, test projects, first arch tests	Clean Architecture, modular monolith, nullable, arch tests, xUnit	1–2 days
2	CI: build and test	GitHub Actions workflow on every PR	CI/CD, GitHub Actions	½–1 day
3	City domain	`City` entity, `Result` / `Error`, domain tests	Entity, invariants, Result pattern, TDD, Guid v7	1 day
4	Application use cases	`CreateCity`, `ListCities`, `ICityRepository`, tests with a fake	Interfaces, CQRS-lite, repository per aggregate, async + CancellationToken, LINQ	1 day

5	Persistence	DbContext, mapping, collation, migration, compose PostgreSQL, Migrator, integration tests	EF Core, migrations, schema/constraints, collations, Testcontainers, configuration	2–3 days
6	API	Endpoints, validation, ProblemDetails, OpenAPI, health checks, JSON logs	DI, minimal APIs, middleware, validation, ProblemDetails, OpenAPI, health checks, logging	2 days
7	Sign-in (API)	Keycloak in compose, JWT bearer, `admin` role, 401/403 tests	OAuth 2 / OIDC / JWT, authorization, secrets	2 days
8	Front-end scaffold	pnpm workspace, Vite app, router, TanStack Query, generated client	TypeScript, pnpm, Vite, components, hooks, router, TanStack Query, generated client	2 days
9	Cities page + browser sign-in	Table + form, OIDC login, token on API calls, front-end tests	OIDC in the browser, component library, forms, Vitest	2–3 days
10	Containers	Dockerfiles (API, Migrator, web), compose runs everything	Containers, multi-stage builds, compose, runtime config	1–2 days
11	Helm on kind	Chart: Deployments, Services, Ingress/Gateway, Secret, probes, migration Job	Pods, Deployments, Services, Ingress, ConfigMap/Secret, probes, resources, namespaces, Helm, Jobs	3–4 days
12	Test cluster + CD	Cloud cluster + managed PG, registry, cert-manager, deploy workflow	Registry & tags, cert-manager, rolling updates, managed services, OIDC to cloud	3–5 days

Total: roughly 4–6 weeks of focused work for a first-time path. Most of the time goes into tasks 5, 7, 11 and 12. That is the point of a walking skeleton: these costs are paid once.

D4. Task by task

Task 0 · Repository housekeeping

```
gh auth login
gh repo create agency --private --source . --remote origin
git branch -m master main && git push -u origin main
# .gitignore additions
._*
.DS_Store
TestResults/
*.trx
coverage/
```

Also add `.editorconfig` (C# style rules the build can enforce) and the empty folders from D2. **Check:** the repository appears on GitHub, with no `._*` files in `git status`.

Task 1 · Solution and architecture tests

```
cd backend
dotnet new sln -n Agency                        # creates Agency.slnx on .NET 10
dotnet new classlib -o src/Agency.Domain
dotnet new classlib -o src/Agency.Application
dotnet new classlib -o src/Agency.Infrastructure
dotnet new web      -o src/Agency.BackOffice    # minimal API host
dotnet new console  -o src/Agency.Migrator
# references point inward only
dotnet add src/Agency.Application    reference src/Agency.Domain
dotnet add src/Agency.Infrastructure reference src/Agency.Application
dotnet add src/Agency.BackOffice     reference src/Agency.Application src/Agency.Infrastructure
dotnet add src/Agency.Migrator       reference src/Agency.Infrastructure
```

```
<!-- backend/Directory.Build.props: applies to every project -->
<Project>
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>
</Project>
```

```
// tests/Agency.ArchitectureTests — the rule written as a test (NetArchTest-style sketch)
[Fact]
public void Domain_does_not_depend_on_frameworks()
{
    var result = Types.InAssembly(typeof(City).Assembly)
        .ShouldNot().HaveDependencyOnAny("Microsoft.EntityFrameworkCore", "Microsoft.AspNetCore")
        .GetResult();
    Assert.True(result.IsSuccessful, string.Join(", ", result.FailingTypeNames ?? []));
}
```

Other first rules: Application does not depend on Infrastructure. Domain and Application do not depend on any host. **Check:** add a forbidden reference on purpose and watch the test fail, then remove it. Update the test command in `CLAUDE.md` to `Agency.slnx`.

Task 2 · CI: build and test on every pull request

```yaml
# .github/workflows/ci.yml (sketch)
name: ci
on:
  pull_request:
  push:
    branches: [main]
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4          # use the current major versions
      - uses: actions/setup-dotnet@v4
        with: { global-json-file: backend/global.json }
      - run: dotnet restore backend/Agency.slnx
      - run: dotnet build backend/Agency.slnx --no-restore -c Release
      - run: dotnet test backend/Agency.slnx --no-build -c Release
```

On GitHub, protect `main`: require the `ci` check to pass before merging. Docker is available on `ubuntu-latest` runners, so Testcontainers integration tests (task 5) work there too. **Check:** open a PR with a failing test and confirm the merge button is blocked.

Task 3 · City domain (test first)

```
// tests/Agency.Domain.Tests/Catalogue/CityTests.cs — written BEFORE City exists
public class CityTests
{
    [Theory]
    [InlineData("")]
    [InlineData("   ")]
    public void Create_rejects_empty_name(string name)
    {
        var result = City.Create(name);
        Assert.True(result.IsFailure);
        Assert.Equal(CityErrors.NameRequired, result.Error);
    }

    [Fact]
    public void Create_trims_the_name()
    {
        var result = City.Create("  Marrakech ");
        Assert.Equal("Marrakech", result.Value.Name);
    }

    [Fact]
    public void Create_rejects_names_longer_than_100_characters()
        => Assert.Equal(CityErrors.NameTooLong, City.Create(new string('a', 101)).Error);
}
```

```
// src/Agency.Domain/Catalogue/City.cs — the entity guards its own rules
public sealed class City
{
    public const int NameMaxLength = 100;
    public Guid Id { get; private set; }
    public string Name { get; private set; } = null!;

    private City() { }                                  // for EF Core materialisation

    public static Result<City> Create(string? name)
    {
        var trimmed = name?.Trim() ?? "";
        if (trimmed.Length == 0)            return CityErrors.NameRequired;
        if (trimmed.Length > NameMaxLength) return CityErrors.NameTooLong;
        return new City { Id = Guid.CreateVersion7(), Name = trimmed };
    }
}

public static class CityErrors
{
    public static readonly Error NameRequired = new("city.name_required", "City name is required.");
    public static readonly Error NameTooLong  = new("city.name_too_long", "City name is at most 100
characters.");
    public static readonly Error NameTaken    = new("city.name_taken", "A city with this name
already exists.");
}
```

WHY IS "UNIQUE" NOT CHECKED HERE?

An entity sees only itself. It cannot know which other cities exist. Uniqueness is a rule across many rows, so it is enforced in two places. The **use case** checks first, to give a friendly error. The **database unique index** guarantees it even when two requests arrive at the same moment (task 5). This "check plus constraint" pattern comes back for contract versions and invoice numbers.

Task 4 · Application: CreateCity and ListCities

```
// src/Agency.Application/Catalogue/ICityRepository.cs — declared here, implemented in
Infrastructure
public interface ICityRepository
{
    Task<bool> NameExistsAsync(string name, CancellationToken ct);
    void Add(City city);
    Task<Result> SaveChangesAsync(CancellationToken ct);       // turns a unique-violation into
NameTaken
}

// src/Agency.Application/Catalogue/CreateCity.cs — a plain handler class, no MediatR
public sealed record CreateCityCommand(string Name);

public sealed class CreateCityHandler(ICityRepository cities, ILogger<CreateCityHandler> logger)
{
    public async Task<Result<Guid>> HandleAsync(CreateCityCommand command, CancellationToken ct)
    {
        var created = City.Create(command.Name);
        if (created.IsFailure) return created.Error;

        if (await cities.NameExistsAsync(created.Value.Name, ct))
            return CityErrors.NameTaken;

        cities.Add(created.Value);
        var saved = await cities.SaveChangesAsync(ct);
        if (saved.IsFailure) return saved.Error;

        logger.LogInformation("City {CityId} created", created.Value.Id);
        return created.Value.Id;
    }
}
```

`ListCities` is a **query**. It only reads, returns a small DTO (`CityDto(Guid Id, string Name)`) sorted by name, and does not load entities for change tracking. Test the handler with a hand-written `FakeCityRepository` (a list in memory). No mocking library is needed.

Task 5 · Persistence

```
// Infrastructure: mapping lives here, not in the entity
internal sealed class CityConfiguration : IEntityTypeConfiguration<City>
{
    public void Configure(EntityTypeBuilder<City> b)
    {
        b.ToTable("cities", "catalogue");                // one PostgreSQL schema per module
        b.HasKey(c => c.Id);
        b.Property(c => c.Name).HasMaxLength(City.NameMaxLength).UseCollation("ci_ai");
        b.HasIndex(c => c.Name).IsUnique();
    }
}
// in AgencyDbContext.OnModelCreating
modelBuilder.HasCollation("ci_ai", locale: "und-u-ks-level1", provider: "icu", deterministic:
false);
```

```
# docker-compose.yml (excerpt) — local only; the password is a dev value, never reused
services:
  postgres:
    image: postgres:17            # or 18: match what the managed service will run
    environment: { POSTGRES_DB: agency, POSTGRES_USER: agency, POSTGRES_PASSWORD: dev_only }
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
volumes: { pgdata: {} }

# create the first migration (tool: dotnet tool install --global dotnet-ef)
dotnet ef migrations add InitialCatalogue -p src/Agency.Infrastructure -s src/Agency.Migrator
dotnet run --project src/Agency.Migrator          # applies migrations, then exits
```

Integration tests (Testcontainers) start a real PostgreSQL, apply migrations, and prove three things: a city round-trips, inserting "fès" after "Fès" violates the unique index, and the repository turns that violation (`SqlState 23505`) into `CityErrors.NameTaken`. **Check:** open DBeaver and look at `catalogue.cities`, its primary key, its unique index and the collation.

Task 6 · API

```
// src/Agency.BackOffice/Endpoints/CityEndpoints.cs
public static class CityEndpoints
{
    public static RouteGroupBuilder MapCityEndpoints(this IEndpointRouteBuilder app)
    {
        var group = app.MapGroup("/api/cities").WithTags("Cities").RequireAuthorization("Admin");

        group.MapGet("/", async (ListCitiesHandler h, CancellationToken ct) =>
            TypedResults.Ok(await h.HandleAsync(ct)));

        group.MapPost("/", async Task<Results<Created<CityCreatedResponse>, ProblemHttpResult>>
            (CreateCityRequest req, CreateCityHandler h, CancellationToken ct) =>
        {
            var result = await h.HandleAsync(new CreateCityCommand(req.Name), ct);
            return result.IsSuccess
                ? TypedResults.Created($"/api/cities/{result.Value}", new
CityCreatedResponse(result.Value))
                : result.Error.ToProblem();            // NameTaken → 409, NameRequired → 400
        });
        return group;
    }
}
public sealed record CreateCityRequest([property: Required, MaxLength(100)] string Name);
```

```
// Program.cs (excerpt) — the host wires everything together
var builder = WebApplication.CreateBuilder(args);
builder.Logging.AddJsonConsole();
builder.Services.AddProblemDetails();
builder.Services.AddValidation();
builder.Services.AddOpenApi();
builder.Services.AddInfrastructure(builder.Configuration);   // DbContext, repositories
builder.Services.AddApplication();                            // handlers
builder.Services.AddHealthChecks().AddDbContextCheck<AgencyDbContext>(tags: ["ready"]);

var app = builder.Build();
app.UseExceptionHandler();                                    // unhandled → 500 ProblemDetails
app.UseAuthentication(); app.UseAuthorization();
app.MapOpenApi();
app.MapHealthChecks("/health/live", new() { Predicate = _ => false });          // process alive
app.MapHealthChecks("/health/ready", new() { Predicate = c => c.Tags.Contains("ready") });
app.MapCityEndpoints();
app.Run();
```

Also add an `.http` file (`backend/requests/cities.http`) to call the endpoints from VS Code or Rider. **Check:** the OpenAPI document lists both endpoints with their 201/400/409 responses, and the logs come out as one JSON object per line.

Task 7 · Sign-in on the API

Run Keycloak in compose. Create a realm `agency`, a public client `backoffice` (authorization code + PKCE), a realm role `admin` and a test user. Export the realm to `deploy/keycloak/agency-realm.json` so anyone can recreate it.

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        o.Authority = builder.Configuration["Auth:Authority"];   // e.g.
http://localhost:8081/realms/agency
        o.Audience  = builder.Configuration["Auth:Audience"];    // the API's name in the token
        o.MapInboundClaims = false;                              // keep claim names as issued
    });
builder.Services.AddAuthorizationBuilder()
    .AddPolicy("Admin", p => p.RequireAuthenticatedUser().RequireRole("admin"));
```

GOTCHA · KEYCLOAK ROLES

By default Keycloak puts realm roles inside a nested claim (`realm_access.roles`), and ASP.NET Core's `RequireRole` will not see them. Fix it with a Keycloak "protocol mapper" that emits a flat `roles` claim (and set `RoleClaimType = "roles"`), or with a small claims transformation in the API. Write an integration test that proves an admin gets 200, a non-admin 403 and an anonymous call 401.

Task 8 · Front-end scaffold

```
cd frontend
pnpm init                                       # root package.json (private)
printf 'packages:\n  - "apps/*"\n  - "packages/*"\n' > pnpm-workspace.yaml
pnpm create vite apps/backoffice --template react-ts
pnpm -C apps/backoffice add @tanstack/react-query react-router
# generate types from the API's OpenAPI document
pnpm -C packages/api-client exec openapi-typescript http://localhost:5080/openapi/v1.json -o
src/schema.d.ts
```

In `vite.config.ts`, proxy `/api` to the local API so the browser sees one origin (decision 11). **Check:** `pnpm -C frontend -r test` and `pnpm -C frontend -r build` both pass, and CI gets a `frontend` job.

Task 9 · Cities page and browser sign-in

```
export function CitiesPage() {
  const queryClient = useQueryClient();
  const cities = useQuery({ queryKey: ["cities"], queryFn: () => api.listCities() });
  const create = useMutation({
    mutationFn: (name: string) => api.createCity({ name }),
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ["cities"] }),  // refetch the list
  });
  if (cities.isPending) return <Spinner />;
  if (cities.isError) return <ErrorBanner error={cities.error} />;
  return (
    <>
      <CityForm onSubmit={create.mutate} error={create.error} />   {/* shows 409 "name taken" */}
      <CityTable rows={cities.data} />
    </>
  );
}
```

Sign-in uses an OIDC library (for example `oidc-client-ts` with `react-oidc-context`). Unauthenticated users are redirected to Keycloak. The access token is attached to every API call. Test the page with Vitest + Testing Library, using a fake API.

Task 10 · Containers

```
# Dockerfile for Agency.BackOffice (multi-stage sketch; run from backend/)
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish src/Agency.BackOffice -c Release -o /app

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS final       # runtime only: much smaller
WORKDIR /app
COPY --from=build /app .
USER $APP_UID                                            # run as non-root
EXPOSE 8080
ENTRYPOINT ["dotnet", "Agency.BackOffice.dll"]
```

Write the same kind of file for the Migrator, plus an nginx image that serves the built SPA with an SPA fallback (`try_files $uri /index.html`). `docker compose up` then runs PostgreSQL, Keycloak, the migrator, the API and the web app together.

GOTCHA · BUILD ONCE, CONFIGURE AT RUNTIME

Vite bakes `import.meta.env.VITE_*` values into the JavaScript at **build** time. If the identity provider URL is baked in, you would need a different image per environment, which breaks "the same image goes from test to production". Instead, the nginx container serves a small `/config.json` (mounted from a ConfigMap) that the app reads at start-up.

Task 11 · Helm on a local kind cluster

```
deploy/helm/agency/
  Chart.yaml
  values.yaml                 defaults (image names, ports, resources)
  values-local.yaml           kind: local hostnames, in-cluster PostgreSQL for convenience
  values-test.yaml            test cluster: real hostnames, TLS, managed DB secret name
  templates/
    api-deployment.yaml  api-service.yaml
    web-deployment.yaml  web-service.yaml  web-config.yaml (ConfigMap: config.json)
    ingress.yaml (or httproute.yaml)
    migrator-job.yaml           Helm hook: pre-install, pre-upgrade

kind create cluster --name agency --config deploy/kind/cluster.yaml
kind load docker-image agency-api:dev agency-web:dev agency-migrator:dev --name agency
helm upgrade --install agency deploy/helm/agency -n agency-local --create-namespace -f
deploy/helm/agency/values-local.yaml
kubectl -n agency-local get pods -w
```

GOTCHA · MIGRATION JOB AND SECRETS

A Helm `pre-install` hook runs *before* the chart's normal resources are created. If the database Secret is a normal template in the same chart, it does not exist yet when the migration Job starts on first install. Create the Secret outside the chart (by hand or via CD, which is also safer), or make the Secret a hook with a lower hook weight.

Task 12 · Test cluster and continuous deployment

1. Create (in the client's cloud account) a small managed Kubernetes cluster, a managed PostgreSQL and DNS records. Install cert-manager and the chosen ingress/gateway controller.
2. Create the namespace `agency-test` and the Secret holding the connection string (never in Git).
3. `cd.yml`: on merge to `main`, build images, tag them with the commit SHA, push them to the registry, then run `helm upgrade --install … --set image.tag=$SHA -f values-test.yaml`.
4. Give GitHub access to the cloud with **OIDC federation** (a short-lived token per run) instead of storing a long-lived cloud password as a GitHub secret.

THE GATE DEMO (SHOW THIS TO THE CLIENT)

1. Open `https://backoffice.test.<domain>`. The padlock shows a valid certificate.
2. Sign in as the test admin. Sign in as a non-admin and show that the page is refused.
3. Add "Taroudant", then try "taroudant" and show the "already exists" message.
4. Show the row in PostgreSQL, the structured log line, and the CI run that deployed it.
5. Merge a tiny visible change and watch it reach the test environment without touching a server.

PART E

Concept reviews

Every concept slice 1 uses, grouped by area. Read a card just before the task that introduces it (the tag shows which). Then apply it, and write your own 5–10 line note in `docs/learning/`.

Concept index

Architecture Decision Record (ADR) 33
Architecture tests 33
async/await and CancellationToken 34
Authentication vs authorization, roles and policies 44
Branches, commits, pull requests 53
CI/CD with GitHub Actions 52
Clean Architecture 29
Collations and case/accent-insensitive text 42
Component library, data grid, forms 47
Configuration and the Options pattern 36
Container registry and image tags 49
Containers, images, multi-stage builds 49
CQRS-lite: commands and queries 32
Dependency and image scanning 45
Dependency injection (DI) 35
EF Core: DbContext and change tracking 40
Entity, invariant, factory method 30
Front-end tests: Vitest and Testing Library 48
Generated API client 47
Health checks 38
Helm: charts and values 51
Ingress, Gateway API, cert-manager and TLS 51
Input validation vs business rules 37
Interfaces and generics 35
kind: Kubernetes in Docker 52
Kubernetes core objects: Pod, Deployment, Service 49
LINQ 35
Liveness, readiness and startup probes 50
Managed PostgreSQL and Redis 53
Middleware and the request pipeline 37
Migrations and the Migrator host 40
Minimal APIs, route groups, endpoint filters 36
Modular monolith 30
Nullable reference types 34
OAuth 2, OpenID Connect, JWT 43
OIDC sign-in in the browser 48
OpenAPI 36
OWASP Top 10, input validation, output encoding 44
pnpm workspaces and Vite 46
PostgreSQL types: numeric, date, timestamptz, uuid 41
ProblemDetails 37
React Router 47
React: components, props, state, hooks 46
Repository per aggregate 32
Resource requests and limits 50
Result pattern vs exceptions 31
Rolling updates and rollbacks 52
Schema design: keys and constraints 41
Secrets management 44
Structured logging with ILogger 38
TanStack Query 46
Test-first and the test pyramid 32
TimeProvider 38
TypeScript 38
Vertical slice and walking skeleton 29
xUnit and Testcontainers 39

E1. Architecture and ways of working

Vertical slice and walking skeleton Whole project

Build one feature through every layer before starting the next. The very first slice is deliberately trivial but deployed for real.

WHY WE NEED IT HERE

A single developer can easily spend months on "the database" or "the framework" with nothing to show. Slices produce something the client can see every two weeks. The **walking skeleton** (a term from Alistair Cockburn) is a minimal end-to-end path that "walks": a city typed in a browser on the cluster reaches the database. Every later slice reuses that path and only adds business logic.

IN THIS PROJECT

Slice 1 = cities. Slice 2 = one hotel contract with one season and one rate, priced for a stay. Each slice follows B2's order: domain test → domain → application → infrastructure → API → screen → Helm values.

WATCH OUT

Do not gold-plate the skeleton (no generic CRUD framework "to save time later"). Do not skip deployment "until later": finding deployment problems early is the whole point.

Clean Architecture Deep dive Task 1

Organise code in layers where every dependency points inward, toward the business rules. Frameworks and databases are details on the outside.

WHY WE NEED IT HERE

The old system put rules in stored procedures and click handlers, so you could not test a price without a database and a form. We want pricing, contract versions and invoices as plain C# that runs in a millisecond-fast unit test. We also want to swap infrastructure (AI provider, storage, WhatsApp provider) without touching business code.

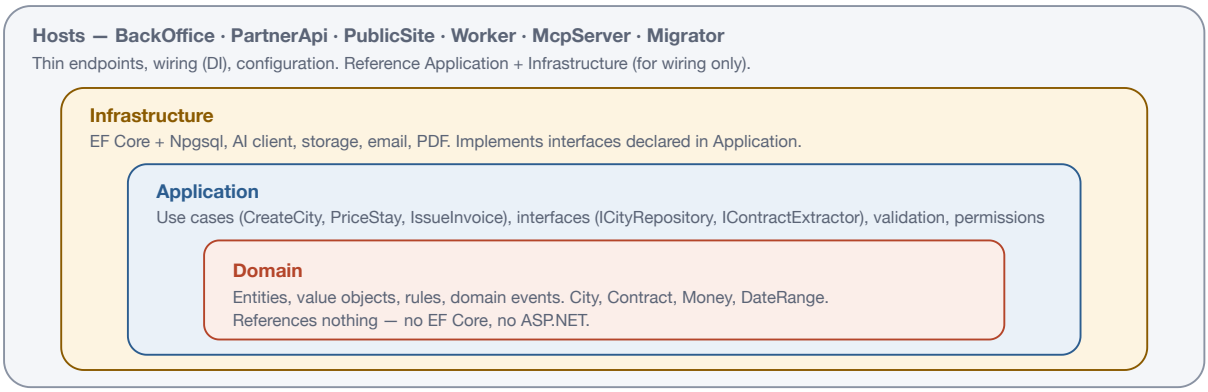


Figure E1 — Each ring may use only rings inside it. Arrows of "uses" always point inward.

HOW IT WORKS: THE TRICK THAT MAKES IT POSSIBLE

The Application layer needs to save a city, but it must not know about EF Core. So Application **declares an interface** (`ICityRepository`), and Infrastructure **implements** it. At start-up the host connects the two through dependency injection. The compile-time dependency points inward (Infrastructure → Application), even though at run time Application calls Infrastructure. This is the **Dependency Inversion Principle**, the "D" in SOLID.

IN THIS PROJECT

Projects `Agency.Domain` → `Agency.Application` → `Agency.Infrastructure` → hosts. Project references make wrong dependencies impossible to compile in one direction, and architecture tests catch the rest (for example a `using Microsoft.EntityFrameworkCore` sneaking into Domain through a package).

WATCH OUT

The common failure is over-layering: five classes and three mappings to list cities. `analysis-and-design.md` warns about this explicitly. Simple reads can be thin queries. Use the full ceremony where rules exist (pricing, billing).

READ

Microsoft's "Common web application architectures" guide (Clean Architecture section). Robert C. Martin, *Clean Architecture*, chapters on the Dependency Rule.

Modular monolith Task 1

One deployable application and one database, divided inside into business modules with clear borders.

WHY WE NEED IT HERE

ADR 0001: one developer and tightly connected rules (a booking needs prices, an invoice needs bookings). Microservices would add network calls, distributed transactions and many deployments for no benefit. The borders still matter, though: without them you get the old system's "everything touches everything".

IN THIS PROJECT

Modules are **folders** in each layer: `Agency.Domain/Catalogue/`, `Agency.Domain/Contracts/`, and so on. A module never uses another module's internals. It calls that module's application interface, or reacts to a domain event. A good habit: give each module its own PostgreSQL **schema** (`catalogue.cities`), so ownership is visible in the database too.

WATCH OUT

"Modular monolith" is not the same as "several hosts". The six hosts (BackOffice, PartnerApi …) are different *entry points* into the same modules. Use `internal` for types that other modules must not use, and add architecture tests for module borders once there are two modules.

Entity, invariant, factory method Task 3

An entity has an identity that stays the same while its data changes. An invariant is a rule it must always satisfy. A factory method is the single door through which valid instances are created.

WHY WE NEED IT HERE

`CLAUDE.md` forbids "anemic models" (classes with public setters and all rules in services). If anyone can write `city.Name = ""`, the rule "name not empty" must be re-checked everywhere, and one place will forget. If the only ways to create or change a city check the rule, an invalid city cannot exist.

IN THIS PROJECT

`City` has a private constructor, private setters and a `static Result<City> Create(string?)`. Later, `Contract` guards "one active version", and `Invoice` refuses changes once issued. Entities are compared by `Id`, not by their data. (Value objects, which are compared by value, such as `Money` and `DateRange`, arrive in slice 2.)

WATCH OUT

EF Core needs a way to create objects when it reads them from the database. A private parameterless constructor is enough, and EF Core can set private setters. Keep the domain free of EF attributes. Mapping goes in Infrastructure configurations.

Result pattern vs exceptions Deep dive Task 3

Return expected failures as a value (`Result`) that the caller must look at. Keep exceptions for bugs and broken infrastructure.

WHY WE NEED IT HERE

"Name already taken", "contract overlaps", "credit limit exceeded" are *normal business outcomes*, not crashes. If they are exceptions, the method signature hides them, callers forget to catch them, and they cost a stack trace each time. A `Result` makes the failure part of the signature: `Task<Result<Guid>>` says "this can fail, deal with it".

HOW IT WORKS

A small type we write ourselves in the Domain (no package needed):

```
public sealed record Error(string Code, string Message);

public class Result
{
    protected Result(Error? error) => Error = error;
    public Error? Error { get; }
    public bool IsSuccess => Error is null;
    public bool IsFailure => !IsSuccess;
    public static Result Success() => new(null);
    public static implicit operator Result(Error error) => new(error);
}

public sealed class Result<T> : Result
{
    private readonly T? _value;
    private Result(T value) : base(null) => _value = value;
    private Result(Error error) : base(error) { }
    public T Value => IsSuccess ? _value! : throw new InvalidOperationException("No value on
failure.");
    public static implicit operator Result<T>(T value) => new(value);
    public static implicit operator Result<T>(Error error) => new(error);
}
```

The implicit operators let you write `return CityErrors.NameTaken;` or `return city;`. At the API edge, one function maps error codes to HTTP: `*.name_taken` → 409 Conflict, validation codes → 400, `*.not_found` → 404.

WATCH OUT

Do not wrap infrastructure errors (database down, timeout) in Results. Those are exceptions, handled once by the global exception handler (→ 500). Reading `.Value` on a failure throws, because it is a bug. The error `Code` is a stable contract with the front end and partners: never rename one casually.

CQRS-lite: commands and queries Task 4

*Use cases that **change** state (commands) are kept separate from use cases that only **read** (queries).*

IN THIS PROJECT

`CreateCityCommand` + `CreateCityHandler` goes through the entity and repository, with rules and logs. `ListCitiesHandler` reads straight into a DTO with `AsNoTracking()` and a projection. Later, reports and lists will be pure queries (even SQL views), while bookings and invoices stay rich commands.

WATCH OUT

"Lite" means one database and plain handler classes. No separate read database, no event sourcing, and no MediatR (`CLAUDE.md` , and MediatR now has a commercial licence). Each handler is a normal class registered in DI and called directly by the endpoint.

Repository per aggregate Task 4

One interface per aggregate root, with only the operations that aggregate needs, written in business words.

WHY

A **generic repository** (`IRepository<T>` with `GetAll` , `Update` , `Delete` for everything) adds code but no value on top of EF Core's `DbSet` , and it invites misuse (deleting an issued invoice). `CLAUDE.md` forbids it. A specific `ICityRepository` with `NameExistsAsync` and `Add` says exactly what is allowed.

AGGREGATE

An aggregate is a cluster of objects saved together through one root, for example `Contract` with its versions, seasons and rates. City is a one-object aggregate. You will meet real aggregates in slice 2.

Test-first and the test pyramid Task 3

Write a failing test that states the rule, make it pass with the simplest code, then clean up ("red → green → refactor"). Have many fast tests at the bottom and fewer slow ones at the top.

Level	Project	What it proves	Speed
Domain unit	`Agency.Domain.Tests`	Business rules: name rules now, pricing later. The most valuable tests in the project.	ms
Application	`Agency.Application.Tests`	Use-case flow with fake repository, fake clock, fake AI	ms
Integration	`Agency.IntegrationTests`	EF mapping, constraints, transactions, real HTTP pipeline + auth, against real PostgreSQL	seconds
Architecture	`Agency.ArchitectureTests`	Layer and module rules	ms
Front end / E2E	Vitest, Playwright	Screens and critical flows	seconds+

WATCH OUT

A test you have never seen fail may be testing nothing. Watch it go red first. Name tests after the rule (`Create_rejects_empty_name`), so a failure reads like a broken business rule.

Architecture tests Task 1

Unit tests that inspect compiled assemblies and fail when a dependency rule is broken.

WHY

Rules in a document get forgotten, and rules in a test fail the build. They guard against you on a tired day and against generated code that "helpfully" adds a reference.

IN THIS PROJECT

Domain has no EF Core or ASP.NET dependency. Application has no Infrastructure dependency. Later: module A does not use module B's internal namespaces, and handlers are `sealed`. Library choice: decision 4 in C3.

Architecture Decision Record (ADR) Task 0

A short file that records one important decision: context, decision, reasons, rejected alternatives. See B5 for the template.

WATCH OUT

ADRs are immutable history. Supersede one, never rewrite it. Keep each to one screen, because nobody reads a long one.

E2. C# and .NET

Nullable reference types Task 1

The compiler tracks which references may be `null` (`string?`) and which may not (`string`), and warns when you mix them up.

IN THIS PROJECT

Turned on for every project in `Directory.Build.props`. With warnings as errors, a possible `NullReferenceException` becomes a build failure. `City.Create(string? name)` admits that input may be null. `City.Name` is `string`, so it is never null once created.

WATCH OUT

`null!` ("trust me, it's not null") is acceptable only where a framework fills the value (EF Core materialisation). Anywhere else it hides a bug.

async/await and CancellationToken Deep dive Task 4

Non-blocking waiting for I/O (database, HTTP, AI). The thread is released while the database works, and resumes when the answer arrives.

HOW IT WORKS

A web server has a limited pool of threads. If each request *blocks* a thread while waiting 20 ms for PostgreSQL, the pool runs out under load. With `await db.SaveChangesAsync(ct)` the method returns a `Task`, and the thread goes back to the pool to serve other requests. When the database answers, the rest of the method continues (possibly on another thread). The compiler rewrites your method into a state machine to make this work.

CANCELLATIONTOKEN

If the user closes the browser tab, ASP.NET Core cancels `HttpContext.RequestAborted`. In minimal APIs, add a `CancellationToken ct` parameter and it is bound automatically. Pass it down to every async call (`ToListAsync(ct)`), so abandoned work stops instead of wasting the database's time. In slice 6 the same mechanism will stop an expensive AI call nobody is waiting for.

RULES

"Async all the way": never call `.Result` or `.Wait()` on a task, because that risks deadlocks and thread starvation. Name async methods `…Async`. Return `Task`, not `async void` (except event handlers). Do not use `Task.Run` to "make things async" in web code.

READ

Microsoft docs: "Asynchronous programming with async and await"; "Cancellation in managed threads".

LINQ Task 4

Query syntax built into C# (`Where`, `Select`, `OrderBy`) that works on in-memory lists and is translated to SQL by EF Core.

```
var cities = await db.Cities.AsNoTracking()
    .OrderBy(c => c.Name)
    .Select(c => new CityDto(c.Id, c.Name))     // only these two columns are read
    .ToListAsync(ct);                            // the SQL runs here, not before
```

WATCH OUT

On a `DbSet`, nothing runs until you materialise (`ToListAsync`, `FirstOrDefaultAsync`, `CountAsync`). Calling `ToList()` too early loads the whole table into memory and filters it in C#. Turn on EF Core's SQL logging in development and read the SQL it generates.

Interfaces and generics Task 4

*An **interface** is a contract ("anything that can do X"). **Generics** let one piece of code work with many types (`Result<T>`, `List<City>`) with full type safety.*

IN THIS PROJECT

`ICityRepository` has a real implementation (EF Core) and a fake one (tests). Later, `IContractExtractor` will have one implementation per AI provider. `Result<T>` is one generic type for every use-case result.

WATCH OUT

Create an interface when there is a real seam (infrastructure, or something a test must replace). Not "one interface per class by habit": that is ceremony.

Dependency injection (DI) Deep dive Task 6

Classes declare what they need in their constructor, and a container builds them, along with their dependencies, when needed.

WHY WE NEED IT HERE

It is the glue of Clean Architecture. `CreateCityHandler` asks for an `ICityRepository`. It does not know (and must not know) that the implementation is `EfCityRepository` using `AgencyDbContext`. The host registers who implements what, once, at start-up.

```
// Agency.Infrastructure/DependencyInjection.cs
public static IServiceCollection AddInfrastructure(this IServiceCollection services,
IConfiguration config)
{
    services.AddDbContext<AgencyDbContext>(o =>
        o.UseNpgsql(config.GetConnectionString("Agency")).UseSnakeCaseNamingConvention());
    services.AddScoped<ICityRepository, EfCityRepository>();
    return services;
}
// Agency.Application/DependencyInjection.cs
services.AddScoped<CreateCityHandler>();
services.AddScoped<ListCitiesHandler>();
```

LIFETIMES: THE PART PEOPLE GET WRONG

Transient: a new instance every time it is requested. **Scoped**: one instance per HTTP request, shared by everything in that request. **Singleton**: one instance for the whole application lifetime. `DbContext` is **scoped**: one unit of work per request. Handlers and repositories using it are scoped too.

WATCH OUT

A **captive dependency** happens when a singleton holds a scoped service (for example a `DbContext`). That context then lives forever, is shared across threads and breaks. In Development, ASP.NET Core validates scopes and throws at start-up when it sees this. Do not "fix" it by making everything a singleton. Avoid resolving services by hand (`GetService`) inside business code; constructor injection keeps dependencies visible.

READ

Microsoft docs: "Dependency injection in ASP.NET Core" and "Service lifetimes".

Configuration and the Options pattern Task 5

Settings come from layered sources, last one wins, and are bound to typed classes that are validated at start-up.

HOW IT WORKS

Default order: `appsettings.json` → `appsettings.{Environment}.json` → user secrets (Development only) → environment variables → command line. In Kubernetes, values arrive as environment variables. A double underscore means "section": `ConnectionStrings__Agency` overrides `ConnectionStrings:Agency`.

```
public sealed class AuthOptions
{
    [Required] public string Authority { get; init; } = "";
    [Required] public string Audience  { get; init; } = "";
}
builder.Services.AddOptions<AuthOptions>()
    .Bind(builder.Configuration.GetSection("Auth"))
    .ValidateDataAnnotations()
    .ValidateOnStart();          // a missing setting stops the pod at start, not at the first
request
```

WATCH OUT

Secrets never go in `appsettings*.json` or Git. Locally use `dotnet user-secrets` or the compose file (with dev-only values). In the cluster use Kubernetes Secrets. `ValidateOnStart` turns a misconfigured deployment into a pod that fails fast and visibly, which Kubernetes reports to you.

Minimal APIs, route groups, endpoint filters Task 6

HTTP endpoints declared as lambdas or static methods (`MapGet`, `MapPost`), without controller classes.

IN THIS PROJECT

`app.MapGroup("/api/cities")` groups endpoints that share a prefix, a tag, and an authorization policy (`RequireAuthorization("Admin")`). Parameters are bound automatically: a record from the JSON body, services from DI, `CancellationToken` from the request. `TypedResults` plus a `Results<Created<…>, ProblemHttpResult>` return type give compile-time checking and let OpenAPI describe every response.

ENDPOINT FILTERS

Code that runs around one endpoint or group (similar to middleware, but with access to the bound arguments). Useful later for cross-cutting checks such as branch scoping.

WATCH OUT

Keep endpoints thin: translate HTTP → command, call the handler, translate Result → HTTP. A business `if` in an endpoint is in the wrong layer.

Middleware and the request pipeline Task 6

A chain of components that every request passes through, in order, and every response passes back through.

IN THIS PROJECT

Order matters: exception handler first (so it catches everything after it), then authentication (who are you?), then authorization (may you?), then the endpoints. Later additions: correlation IDs, rate limiting (partner API), output caching (public site).

WATCH OUT

`UseAuthorization()` before `UseAuthentication()` means nobody is ever authenticated when authorization runs. A classic bug.

ProblemDetails Task 6

A standard JSON shape for HTTP errors (RFC 9457): `type`, `title`, `status`, `detail`, *plus your own fields.*

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json
{ "type": "https://agency.example/errors/city.name_taken", "title": "A city with this name
already exists.",
  "status": 409, "code": "city.name_taken", "traceId": "00-4bf9…" }
```

WHY

One error format for the back office, the partners and the AI tools. The front end can show `title` and branch on `code`. `AddProblemDetails()` + `UseExceptionHandler()` also turn unhandled exceptions into a 500 ProblemDetails *without* leaking the stack trace.

Input validation vs business rules Task 6

Input validation checks the shape of a request (required, length, format). Business rules check what the domain allows.

IN THIS PROJECT

The request record has `[Required, MaxLength(100)]`, and .NET 10's built-in validation returns a 400 ValidationProblem before your handler even runs. `City.Create` checks again, because the domain must never trust its caller (the Worker or MCP host could call it too). Uniqueness is a business rule (409), not an input rule (400).

WATCH OUT

Duplicating the length check in two places is fine. Moving the business rule out of the domain into validation attributes is not.

Structured logging with ILogger Task 6

Logs as events with named fields, not just text, so you can search "all logs where CityId = …".

```
logger.LogInformation("City {CityId} created", city.Id);    // ✓ template + field
logger.LogInformation($"City {city.Id} created");           // ✗ string interpolation: field
lost
```

IN THIS PROJECT

`AddJsonConsole()` writes one JSON object per line to stdout, and Kubernetes collects stdout. Levels: `Information` for business events, `Warning` for unexpected but handled situations, `Error` for failures. Never log passwords, tokens or guest personal data (names, passports). Log IDs instead.

OpenAPI Task 6

A machine-readable description of the API (paths, request and response shapes, error codes) in JSON or YAML.

IN THIS PROJECT

.NET 10 generates it (`AddOpenApi()` / `MapOpenApi()` → `/openapi/v1.json`, OpenAPI 3.1). It feeds three things: the interactive UI (Scalar or Swagger UI) to try endpoints, the **generated TypeScript client** (task 8), and later the partner documentation. Part of the definition of done is "OpenAPI updated". The front end's type check fails if the API changed and the client was not regenerated, which is exactly what you want.

Health checks Task 6

Endpoints that report whether the app is alive and whether it is ready to serve, for Kubernetes probes.

IN THIS PROJECT

`/health/live` checks no dependencies: it answers if the process is not stuck. `/health/ready` includes the database check. See the "probes" card (E6) for why the two must differ.

TimeProvider From slice 2, rule now

An injectable clock (`TimeProvider.System` in production, `FakeTimeProvider` in tests) instead of `DateTime.Now`.

WHY

Release days, option expiry and "early booking until 31 March" all depend on "today". Tests must control today. `DateTime.Now` is also local time, while we store UTC (`DateTimeOffset` / `timestamptz`). If slice 1 records a `CreatedAt`, take it from `TimeProvider.GetUtcNow()`.

xUnit and Testcontainers Tasks 1, 5

xUnit runs tests (`[Fact]` = one case, `[Theory]` + `[InlineData]` = the same test with several inputs). Testcontainers starts real services (PostgreSQL) in Docker for a test run, then removes them.

WHY REAL POSTGRESQL

EF Core's in-memory provider does not enforce unique indexes, collations or exclusion constraints, and those are exactly what we need to prove. A shared container per test class (xUnit "fixture") keeps tests fast.

WATCH OUT

Integration tests need Docker running locally and in CI. Each test must create its own data or clean up. Never depend on test order.

E3. Data: EF Core and PostgreSQL

EF Core: DbContext and change tracking Deep dive Task 5

An ORM (object-relational mapper) that maps C# classes to tables, translates LINQ to SQL, and tracks changes to loaded objects so that `SaveChanges` writes them.

HOW IT WORKS

`AgencyDbContext` has one `DbSet<City>` per aggregate root. When you load an entity, the context's **change tracker** keeps a snapshot. When you call `SaveChangesAsync`, it compares, generates `INSERT` / `UPDATE` / `DELETE` statements, and runs them **in one transaction**. That makes the DbContext a natural **unit of work**: everything changed during one request is saved together or not at all.

IN THIS PROJECT

The mapping lives in `IEntityTypeConfiguration<T>` classes in Infrastructure (the Domain stays clean). The Npgsql provider talks to PostgreSQL. `UseSnakeCaseNamingConvention()` turns `City.Name` into column `name`. Queries that only read use `AsNoTracking()` (faster, no snapshot).

WATCH OUT

- A DbContext is not thread-safe. One per request (scoped), and never two queries at once on the same context.
- Translating database errors: a unique violation arrives as `DbUpdateException` wrapping a `PostgresException` with `SqlState == "23505"`. The repository catches exactly that and returns `CityErrors.NameTaken`. Everything else stays an exception.
- Watch the SQL that LINQ generates. N+1 queries (one query per row in a loop) are the classic ORM performance bug.

READ

learn.microsoft.com/ef/core (Getting started, Change tracking, Saving data) and npgsql.org/efcore.

Migrations and the Migrator host Deep dive Task 5

Versioned, code-reviewed steps that move the database schema from one version to the next, generated by comparing your model with a snapshot.

HOW IT WORKS

`dotnet ef migrations add InitialCatalogue` compares the current model with `AgencyDbContextModelSnapshot.cs` and writes a migration class with `Up()` (apply) and `Down()` (revert). The database records applied migrations in `__EFMigrationsHistory`. Applying runs the missing ones in order.

WHY A SEPARATE MIGRATOR

With 2+ API pods starting at the same time, "migrate at start-up" means several pods race to change the schema, and a slow migration delays every pod's readiness. Instead, `Agency.Migrator` is a small console app that applies migrations and exits, run **once per release** as a Kubernetes Job *before* the new pods start (ADR 0001, design doc "Running on Kubernetes").

WATCH OUT

- **Read every generated migration** before committing. It is code that changes production data.
- Never edit a migration that has run anywhere shared. Add a new one.
- Schema changes must be **backward compatible** with the previous app version for a moment, because old pods still run during a rolling update. Example: add a column as nullable, deploy, backfill, then make it required in a later release ("expand → contract").
- For things EF cannot express (exclusion constraints in slice 2), use `migrationBuilder.Sql(...)` inside the migration.

Schema design: keys and constraints Task 5

Let the database itself refuse bad data: primary keys, foreign keys, `NOT NULL`, `UNIQUE`, `CHECK`, *and later* `EXCLUDE`.

WHY

The old database had 112 tables without a primary key and no foreign keys, so duplicates and orphans crept in. Code checks can be bypassed (a bug, a script, two concurrent requests). Constraints cannot. The rule is **check in code for a friendly message, constrain in the database for the guarantee**.

IN THIS PROJECT

`catalogue.cities`: `id uuid PRIMARY KEY`, `name varchar(100) NOT NULL COLLATE ci_ai`, `UNIQUE (name)`. Later: `CHECK (amount >= 0)`, foreign keys everywhere, exclusion constraints on `daterange` for seasons and contract versions.

PostgreSQL types: numeric, date, timestamptz, uuid Task 5

C#	PostgreSQL	Use for
`decimal`	`numeric(18,2)`	Money. Exact. `float` / `double` are binary approximations: 0.1 + 0.2 ≠ 0.3.
`decimal`	`numeric(18,6)`	Exchange rates.
`DateOnly`	`date`	Stay dates, season limits: a calendar day, no time zone.
`DateTimeOffset` (UTC)	`timestamptz`	Moments: created at, approved at. Npgsql requires UTC.
`Guid`	`uuid`	Identifiers (decision 6).

Collations and case/accent-insensitive text Task 5

A collation decides how text is compared and sorted. PostgreSQL compares case-sensitively by default, so "Fès" ≠ "fès" ≠ "Fes".

IN THIS PROJECT

Hotel, client and city names are French or Moroccan and full of accents. A **non-deterministic ICU collation** `und-u-ks-level1` compares at "primary strength": it ignores case and accents. Put a unique index on a column with this collation and the database refuses "fes" when "Fès" exists. The stored value keeps its original spelling.

WATCH OUT

Non-deterministic collations have limits (some pattern matching, some index types). For searching, add a separate index later. `citext` is the older alternative, but it ignores case only, not accents.

E4. Security

OAuth 2, OpenID Connect, JWT

Deep dive · Tasks 7, 9

*OAuth 2 lets an application get an **access token** to call an API. OpenID Connect (OIDC) adds **sign-in** on top, meaning who the user is. A JWT is the token format: signed JSON the API can verify by itself.*

WHY WE NEED IT HERE

Staff, partners (by API), the MCP client and later guests all need to prove who they are, and the API must decide what each may do. We do not want to store passwords or write login screens ourselves. A dedicated **identity provider** (Keycloak, Entra ID) does that, and our API only checks tokens.

THE FLOW IN SLICE 1: AUTHORIZATION CODE + PKCE

1. The user opens the back office. The SPA sees no session and redirects the browser to Keycloak's login page, with a random **code verifier** hashed into a **code challenge** (that is PKCE).
2. The user signs in *on Keycloak*. Our app never sees the password.
3. Keycloak redirects back with a one-time **authorization code**.
4. The SPA exchanges the code plus the original verifier for tokens. PKCE proves that whoever started the login is the one finishing it, so a stolen code is useless.
5. The SPA calls `/api/cities` with `Authorization: Bearer <access token>`.
6. The API checks the token's **signature** (with Keycloak's public keys, downloaded once from `/.well-known/openid-configuration`), its **issuer**, its **audience** (is it meant for this API?) and its **expiry**, then reads the roles.

INSIDE A JWT

Three base64url parts: `header.payload.signature`. The payload holds claims such as `sub` (user ID), `iss`, `aud`, `exp` and `roles`. It is **signed, not encrypted**: anyone can read it (try jwt.io with a test token), but nobody can change it without breaking the signature. Never put secrets in it.

OTHER FLOWS YOU WILL MEET

Client credentials (machine to machine, no user): partners calling the partner API, phase 5. The MCP server uses OAuth for remote clients (phase 5–6).

WATCH OUT

Access tokens are short-lived (minutes). Refresh happens through the OIDC library. Tokens in `localStorage` can be stolen by any cross-site scripting bug, so prefer memory or session storage, or a BFF (decision 10). Always validate `aud`, or a token issued for another app would be accepted. Never accept unsigned tokens (`alg: none`).

READ

oauth.net/2 (overview of flows), "OAuth 2.0 for Browser-Based Apps" (IETF), the Microsoft docs on JWT bearer authentication in ASP.NET Core, and Keycloak's "Getting started with Docker".

Authentication vs authorization, roles and policies Task 7

Authentication answers "who are you?" (a valid token gives an identity). Authorization answers "may you do this?" (policies checked per endpoint).

IN THIS PROJECT

Slice 1 has one policy, `Admin` (role `admin`). From slice 2 there will be named **permissions** such as `ApproveContract` and `IssueInvoice`, plus **data scopes** (a branch manager sees one branch). Business code checks the permission, not the role name, so roles can be reorganised without code changes.

WATCH OUT

401 = not authenticated (no or invalid token). 403 = authenticated but not allowed. Hiding a button in React is **not** security. The API enforces every permission (`CLAUDE.md`, AI rules). Test both codes.

Secrets management Tasks 5, 7, 12

Passwords, keys and connection strings live outside the code and outside Git, and reach the app at run time.

IN THIS PROJECT

Locally: `dotnet user-secrets`, or dev-only values in compose that are never reused. Cluster: Kubernetes Secrets created by CD or by hand, later perhaps synced from a key vault. CI: GitHub encrypted secrets, and better, OIDC federation to the cloud (no stored cloud password at all). This directly fixes the old system's plain-text SQL and Gmail passwords.

WATCH OUT

Kubernetes Secrets are only base64-encoded, not encrypted. Anyone with read access to the namespace can read them, so limit access. If a secret is ever committed, **rotate it**: deleting the commit is not enough. Turn on GitHub's secret scanning and push protection.

OWASP Top 10, input validation, output encoding Whole project

The OWASP Top 10 (latest edition 2025) lists the most common web vulnerability categories. Use it as a review checklist.

THE ONES THAT MATTER MOST HERE

Broken access control: an endpoint without a policy, or a partner reading another partner's booking by changing an ID. Test both. **Injection**: EF Core parameterises queries. Never build SQL with string concatenation, even in `FromSqlRaw`. **Security misconfiguration**: Swagger UI or detailed errors in production, default passwords. **Vulnerable or compromised dependencies** (supply chain): see scanning below. **Authentication failures**: handled by the identity provider. **Logging failures**: log security events (failed logins, 403s), but never secrets.

OUTPUT ENCODING

React escapes text by default. Never use `dangerouslySetInnerHTML` with user or AI content. AI output is untrusted input too (prompt injection, phase 6).

Dependency and image scanning Tasks 2, 12

Automatically look for known vulnerabilities (CVEs) in the packages and container images you ship.

IN THIS PROJECT

Dependabot (GitHub) opens PRs for vulnerable or outdated packages: NuGet, npm, GitHub Actions and Docker base images. In CI, `dotnet list package --vulnerable` and `pnpm audit` for packages, and an image scanner (for example Trivy) on built images. Fail the build on high or critical findings.

E5. Front end

TypeScript Task 8

JavaScript plus static types, checked at build time. Types disappear at run time.

IN THIS PROJECT

`strict: true` in `tsconfig.json` (the front-end equivalent of nullable reference types). API types are **generated** from OpenAPI, never written by hand, so `city.name` is known to be a `string`, and the build breaks when the API changes. Learn: `type` vs `interface`, union types (`"pending" | "error" | "success"`), generics, narrowing with `if`.

WATCH OUT

`any` switches checking off. Treat it like `null!`. Types do not validate data at run time: data from outside (API, forms) is validated with a schema library such as Zod (phase 2).

pnpm workspaces and Vite Task 8

pnpm installs packages fast with a shared store. Workspaces hold several apps and packages in one repository. Vite is the dev server (instant reload) and the production bundler.

IN THIS PROJECT

`frontend/apps/backoffice` (later `partner-portal`, `public-site`) and `frontend/packages/api-client`, `packages/ui`. An app depends on a package with `"@agency/api-client": "workspace:*"`. `pnpm -C frontend -r test` runs tests in every package. Commit `pnpm-lock.yaml` so every install, including CI's, is identical.

React: components, props, state, hooks Tasks 8, 9

*A UI is a tree of **components**: functions that take **props** (inputs) and return markup. **State** is data that, when it changes, makes React re-render. **Hooks** (`useState`, `useEffect`, custom `useXxx`) let a component use state and other React features.*

```
function CityForm({ onSubmit }: { onSubmit: (name: string) => void }) {
  const [name, setName] = useState("");                    // state: re-render on change
  return (
    <form onSubmit={e => { e.preventDefault(); onSubmit(name); setName(""); }}>
      <input value={name} onChange={e => setName(e.target.value)} />  {/* controlled input */}
      <button type="submit">Add city</button>
    </form>
  );
}
```

MENTAL MODEL

UI = f(state). You never change the screen directly. You change state, and React computes the new UI. Data flows **down** through props, and events flow **up** through callback props (`onSubmit`).

WATCH OUT

Hooks only at the top level of a component (never inside `if` or loops). `useEffect` is for synchronising with outside systems. Do not use it to fetch data by hand (TanStack Query does that) or to compute values derivable from props. Never mutate state objects. Create new ones.

READ

react.dev/learn ("Thinking in React", "Managing state", "You might not need an effect").

React Router Task 8

Maps URLs to pages in a single-page app, without full page reloads.

IN THIS PROJECT

`/cities` → `CitiesPage`, inside a layout with navigation. Later, **route guards** redirect to sign-in or hide pages the user's permissions do not allow (phase 2). Because it is an SPA, the web server must answer *every* unknown path with `index.html` (nginx `try_files`), or refreshing `/cities` returns a 404.

TanStack Query Deep dive Task 9

*A library that fetches, caches, deduplicates and refreshes **server state** (data that lives on the server), so components just declare what they need.*

WHY

Hand-written `useEffect` + `fetch` means writing loading flags, error handling, race conditions, refetching and cache invalidation in every screen. TanStack Query does all of that the same way everywhere, and the back office will have hundreds of lists.

KEY IDEAS

Query key (`["cities"]`, later `["contracts", id]`): the cache address. Same key = shared data and one request. **Stale time**: how long data counts as fresh before a background refetch. **Mutations** (`useMutation`) change data. On success, **invalidate** the affected keys and the lists refetch themselves (see task 9). Query state is explicit: `isPending`, `isError`, `data`.

WATCH OUT

Do not copy query data into `useState`, because you then have two sources of truth. Design query keys like URLs: hierarchical and consistent. Keep one `QueryClient` for the app, and a fresh one per test.

Generated API client Task 8

TypeScript types (and optionally functions) generated from the API's OpenAPI document.

IN THIS PROJECT

`packages/api-client`: `openapi-typescript` generates `schema.d.ts`, and `openapi-fetch` gives a typed `client.GET("/api/cities")`. A script `pnpm gen:api` regenerates it, and CI checks that the committed client matches the API (regenerate, then `git diff --exit-code`). Result: a renamed field in C# becomes a TypeScript compile error, not a bug in production.

Component library, data grid, forms Task 9

Ready-made, accessible inputs, tables, dialogs and layouts, so you build business screens, not buttons.

IN THIS PROJECT

Decision 14 (MUI / Ant Design / shadcn/ui). The back office is table-heavy (contracts, rates, bookings), so pick one with a strong data grid: sorting, filtering, paging, and virtualisation for long lists. Forms: React Hook Form + Zod from phase 2, when forms get big.

OIDC sign-in in the browser Task 9

An OIDC client library runs the authorization code + PKCE flow (card E4) inside the SPA.

IN THIS PROJECT

`oidc-client-ts` (with `react-oidc-context`): an `AuthProvider` at the root, a guard that triggers sign-in, and an API client middleware that adds `Authorization: Bearer …`. The identity provider URL and client ID come from the runtime `/config.json` (task 10 gotcha), not from build-time env vars.

Front-end tests: Vitest and Testing Library Task 9

Vitest runs tests (Jest-compatible, Vite-native). Testing Library renders components and queries them the way a user would ("the button named Add city").

IN THIS PROJECT

Test that the cities page shows rows, that submitting calls the API, and that a 409 shows "already exists". Mock the network (for example with MSW, Mock Service Worker), not the components. Playwright end-to-end tests come in phase 2 for critical flows.

E6. Containers, Kubernetes and delivery

Containers, images, multi-stage builds Deep dive Task 10

*An **image** packages an app with everything it needs to run (runtime, libraries, files). A **container** is a running instance of an image, isolated from other processes.*

WHY WE NEED IT HERE

"Works on my machine" disappears: the image that passed tests in CI is *exactly* what runs on test and in production. Kubernetes only runs containers.

HOW IT WORKS

A `Dockerfile` is a recipe of layers. Each instruction adds a cached layer, so put rarely-changing steps (restore packages) before often-changing ones (copy source) to keep rebuilds fast. A **multi-stage** build uses a big SDK image to compile, then copies only the published output into a small runtime image (task 10). The final image holds no compiler, no source, no test code: smaller, faster to pull, and less to attack.

IN THIS PROJECT

Images: `agency-api` (BackOffice host), `agency-migrator`, `agency-web` (nginx + built SPA), and later one per host. .NET images listen on port **8080** by default and include a non-root `app` user. `docker compose` runs the whole system locally with one command. The .NET SDK can also build images without a Dockerfile (`dotnet publish /t:PublishContainer`), but write the Dockerfile once so you understand it.

WATCH OUT

Add a `.dockerignore` (`bin/`, `obj/`, `node_modules/`, `.git`), or the build context becomes huge and may leak files. Never bake secrets into an image: anyone who can pull it can read them. Containers are disposable, so never write important files to the container's disk.

Container registry and image tags Task 12

A registry stores images (GHCR, ACR). A tag names a version of an image.

IN THIS PROJECT

CI pushes `ghcr.io/<org>/agency-api:<git-sha>`. The commit SHA makes every tag unique and traceable to the exact code. Helm deploys that SHA. Avoid deploying `:latest`: you cannot tell what is running, and rollbacks become guesswork.

Kubernetes core objects: Pod, Deployment, Service Deep dive Task 11

Kubernetes runs containers across machines and keeps reality equal to what you declared: "I want 2 copies of agency-api version X, reachable at this name".

DECLARATIVE MODEL

You do not run commands like "start a container". You submit YAML describing the **desired state**. Controllers in the cluster constantly compare desired with actual state and fix the difference: a crashed container is restarted, a lost node's pods are recreated elsewhere.

Object	What it is	In this project
Pod	The smallest unit: one (or a few tightly coupled) containers with a shared IP. Disposable and replaceable at any time.	One API container per pod.

Deployment	Keeps N identical pods running (through a ReplicaSet) and replaces them gradually on a new version.	`api` (2 replicas), `web` (2 replicas).
Service	A stable internal name and IP that load-balances to the pods matching a label selector. Pods come and go, but the Service stays.	`api` → pods labelled `app=api`, port 80 → 8080.
Ingress / Gateway	HTTP(S) routing from outside the cluster to Services, by host name and path. Needs a controller installed.	`/api` → api Service, `/` → web Service, TLS.
ConfigMap / Secret	Configuration given to pods as environment variables or files. Secrets are for sensitive values.	Auth settings, `config.json` for the SPA / connection string.
Job / CronJob	Pods that run to completion, once or on a schedule.	Migrator Job per release. Nightly checks later.
Namespace	A named partition of a cluster: names, permissions and quotas per environment.	`agency-local`, `agency-test`, later `agency-prod`.

WATCH OUT

Pods are cattle, not pets: no state on local disk, any pod can die at any time. That is why ASP.NET Core Data Protection keys must be shared (Redis, phase 3) once you rely on cookies, and why concurrency must be safe in the database. `kubectl describe pod` and `kubectl logs` are your first debugging tools. The **Events** at the bottom of `describe` usually explain the problem (image pull failure, failing probe, out of memory).

READ

kubernetes.io/docs/tutorials ("Learn Kubernetes Basics"), then the Concepts pages for each object.

Liveness, readiness and startup probes Task 11

*Kubernetes calls your health endpoints to decide whether to **restart** a pod (liveness) and whether to **send it traffic** (readiness).*

IN THIS PROJECT

Liveness → `/health/live` (no dependencies). Readiness → `/health/ready` (includes PostgreSQL). If the database blips, pods become *not ready* (no traffic) but are **not restarted**. If liveness checked the database, a database outage would make Kubernetes restart every pod in a loop, which makes the outage worse. A startup probe gives slow starts time before liveness kicks in.

Resource requests and limits Task 11

***Requests** are what the scheduler reserves for a pod. **Limits** are the maximum it may use.*

IN THIS PROJECT

Start modest (for example API: request 100m CPU / 256Mi memory, memory limit 512Mi) and adjust from real metrics. A pod above its memory limit is killed (`OOMKilled` in `describe`). .NET reads the container's memory limit and sizes its garbage collector accordingly. Without requests, the scheduler packs pods blindly.

Helm: charts and values Deep dive Task 11

*A package manager and template engine for Kubernetes YAML. A **chart** is a folder of templates. **Values** files fill them in per environment.*

WHY

Local, test and production need the same objects with different host names, replica counts, image tags and secret names. Copying YAML three times drifts. With Helm: one chart, one values file per environment.

```
# templates/api-deployment.yaml (excerpt)
spec:
  replicas: {{ .Values.api.replicas }}
  template:
    spec:
      containers:
        - name: api
          image: "{{ .Values.image.registry }}/agency-api:{{ .Values.image.tag }}"
          envFrom:
            - secretRef: { name: {{ .Values.database.secretName }} }
          readinessProbe: { httpGet: { path: /health/ready, port: 8080 } }
          livenessProbe:  { httpGet: { path: /health/live,  port: 8080 } }

# values-test.yaml
api: { replicas: 2 }
image: { registry: ghcr.io/your-org, tag: "set-by-ci" }
database: { secretName: agency-db }
```

COMMANDS

`helm template` renders locally (read the output!). `helm upgrade --install` installs or upgrades a **release**. `helm history` and `helm rollback` undo a bad release. **Hooks** (`pre-install`, `pre-upgrade`) run Jobs such as the migrator before the new version rolls out.

WATCH OUT

Template whitespace and indentation errors are common. Run `helm lint` and `helm template` in CI. Hook ordering versus Secrets: see the gotcha in task 11.

Ingress, Gateway API, cert-manager and TLS Tasks 11, 12

Ingress (and its successor, the Gateway API) routes outside HTTP traffic to Services. cert-manager obtains and renews TLS certificates automatically (for example from Let's Encrypt).

IN THIS PROJECT

One host name per front end, `/api` routed to the API (same origin, no CORS). A `ClusterIssuer` for Let's Encrypt. Certificates renew without anyone remembering. See the ingress-nginx retirement box in C2 before choosing a controller.

Rolling updates and rollbacks Task 12

A Deployment replaces old pods with new ones gradually, sending traffic only to pods that are ready, so there is no downtime.

IN THIS PROJECT

This only works if readiness probes are honest, and if old and new versions can run side by side for a moment against the same database schema ("expand → contract" migrations, E3). Undo with `helm rollback`, or by redeploying the previous SHA.

kind: Kubernetes in Docker Task 11

A full Kubernetes cluster running as Docker containers on your laptop. Free, disposable, and identical in API to a cloud cluster.

IN THIS PROJECT

Learn and debug the Helm chart locally before paying for cloud. Images built locally must be loaded into kind (`kind load docker-image`). To reach an Ingress on kind, the cluster config maps ports 80/443 to your machine (`extraPortMappings`). `kind delete cluster` and start again whenever you are lost: that is the point.

CI/CD with GitHub Actions Deep dive Tasks 2, 12

***Continuous integration**: every push is built and tested automatically. **Continuous delivery/deployment**: every merge to the main branch produces images and deploys them to the test environment.*

HOW GITHUB ACTIONS WORKS

A **workflow** (`.github/workflows/*.yml`) is triggered by **events** (`pull_request`, `push`). It contains **jobs** that run on fresh **runners** (virtual machines), and each job is a list of **steps**: shell commands, or reusable **actions** (`actions/checkout`). Jobs run in parallel unless one `needs` another.

IN THIS PROJECT

`ci.yml` on every PR: backend build + tests (including Testcontainers), front-end type check + lint + tests + build, OpenAPI client drift check, Helm lint. `cd.yml` on merge to `main`: build and push images tagged with the SHA, run `helm upgrade --install` against `agency-test`. Branch protection makes the CI check mandatory before merging.

WATCH OUT

- Give the workflow's `GITHUB_TOKEN` only the permissions it needs (`permissions:` block, for example `packages: write` for GHCR).
- Prefer **OIDC federation** to the cloud over storing long-lived cloud credentials as secrets.
- Pin third-party actions to a version (ideally a commit SHA). An action is code that runs with your secrets.
- Cache NuGet and pnpm stores to keep runs fast.
- Keep production deployment behind a manual approval (GitHub "environments") when you get there.

READ

docs.github.com/actions ("Understanding GitHub Actions", "Security hardening for GitHub Actions").

Managed PostgreSQL and Redis Task 12

The cloud provider runs the database server: patching, backups, point-in-time recovery, failover. You get a connection string.

IN THIS PROJECT

ADR 0004: billing data must not be lost, so production PostgreSQL is managed, outside the cluster. For local and kind, a PostgreSQL container is fine because the data is disposable. Before go-live, **test a restore**: a backup you have never restored is a hope, not a backup. Require TLS on the connection and allow only the cluster's network.

E7. Git and pull requests

Branches, commits, pull requests Task 0

Every change is made on a short-lived branch, reviewed as a pull request and merged into `main`, which must always be deployable.

HABITS

Small commits that each build. Messages in the imperative that say *why* ("Add unique index on city name to prevent duplicates under concurrency"). Stage with `git add -p` so you read every hunk. Squash-merge PRs for a clean history. Delete merged branches. Rebase your branch on `main` if it falls behind (`git pull --rebase origin main`).

REVIEWING YOUR OWN PR

Read the diff on GitHub as if a stranger wrote it. Check it against the definition of done (B4). With AI-written code, ask: does it follow `CLAUDE.md`? Did it add a dependency? Did it invent an API that does not exist? Is every new branch of logic tested?

APPENDIX

Reference

F1. Acronyms

Acronym	Meaning	Acronym	Meaning
ADR	Architecture Decision Record	ORM	Object-relational mapper (EF Core)
BFF	Backend for frontend (server keeps tokens for the browser)	PKCE	Proof Key for Code Exchange (protects the OAuth code)
CI/CD	Continuous integration / continuous delivery	RFC	Request for Comments: an internet standard document
CNDP	Moroccan data-protection authority (Law 09-08)	SPA	Single-page application
CORS	Cross-origin resource sharing (browser rule for calls to another origin)	SHA	Here: the Git commit hash used as an image tag
CPM	Central Package Management (`Directory.Packages.props`)	TLS	Transport Layer Security: the "S" in HTTPS
CQRS	Command/query responsibility segregation	TO	Tour operator
DDD	Domain-driven design	DMC	Destination management company
DI	Dependency injection	OTA	Online travel agency
DTO	Data transfer object: a plain shape for crossing a boundary	LTS	Long-term support release
JWT	JSON Web Token	MCP	Model Context Protocol (manager assistant)
OIDC	OpenID Connect	RAG	Retrieval-augmented generation (guest assistant, phase 6)

F2. Official documentation

Topic	Where
.NET, C#	learn.microsoft.com/dotnet
ASP.NET Core (minimal APIs, auth, health, OpenAPI)	learn.microsoft.com/aspnet/core
EF Core / Npgsql provider	learn.microsoft.com/ef/core · npgsql.org/efcore
PostgreSQL (collations, constraints, ranges)	postgresql.org/docs
Testcontainers for .NET	dotnet.testcontainers.org

React · TanStack Query · React Router	react.dev/learn · tanstack.com/query · reactrouter.com
Vite · pnpm · TypeScript	vite.dev · pnpm.io · typescriptlang.org/docs
Docker	docs.docker.com (Dockerfile best practices, multi-stage builds)
Kubernetes · kind · Helm	kubernetes.io/docs/tutorials · kind.sigs.k8s.io · helm.sh/docs
Gateway API · cert-manager	gateway-api.sigs.k8s.io · cert-manager.io/docs
GitHub Actions	docs.github.com/actions
OAuth 2 / OIDC · Keycloak	oauth.net/2 · openid.net · keycloak.org/documentation
OWASP Top 10	owasp.org/www-project-top-ten

F3. What comes next (slice 2 preview)

After the gate demo: **a hotel contract with one season and one rate, and the pricing engine returning a price for a stay.** New concepts then: value objects (`Money`, `DateRange`, `Occupancy`), `decimal` rounding rules, `DateOnly`, `TimeProvider`, contract versions as a state machine, `daterange` + exclusion constraints, optimistic concurrency, the audit log, and the first domain tests that really matter, based on the contracting team's worked examples. This handbook gains a new Part E section at that point.